

Gulp — specyfikacja frameworka gier 2D (Java, desktop + web)

Sep 23, 2026 · @Someone

1. Instrukcje dla Claude

Ten dokument jest jedynym źródłem prawdy dla projektu Gulp: opisuje co zbudować, w jakiej kolejności i kiedy zadanie jest skończone. Każda sesja pracy zaczyna się od przeczytania sekcji 1-8 oraz sekcji dotyczącej bieżącego zadania.

Jak pracować z tym dokumentem

1. Otwórz sekcję 22 (Roadmapa) i weź pierwsze nieodhaczone zadanie z najniższego nieukończonego etapu. Nie przeskakuj etapów.
2. Przeczytaj specyfikację podsystemu, którego dotyczy zadanie (sekcje 8-20). Nazwy klas, metod i pakietów z dokumentu są wiążące, chyba że sekcja 23 (Otwarte decyzje) mówi inaczej.
3. Najpierw napisz interfejsy w `gulp-api` z pełnym Javadoc, potem testy, potem implementację w `gulp-core` lub backendzie.
4. Uruchom `./gradlew check` oraz build webowy (`./gradlew :examples:showcase:buildWeb`). Zadanie nie jest skończone, dopóki oba przechodzą.
5. Zaktualizuj `CHANGELOG.md` i odhacz zadanie w roadmapie. Jeśli podjąłeś decyzję, której dokument nie opisywał, dopisz ją do `docs/decisions/NNNN-tytuł.md` (ADR).
6. Jeśli specyfikacja jest sprzeczna, niejasna albo niewykonalna pod TeaVM, zatrzymaj się i zapytaj użytkownika zamiast zgadywać.

Twarde reguły (nie łamać bez zgody użytkownika)

- Kod gry importuje wyłącznie pakiety `dev.gulp.api.*`. Nic z `core`, `platform` ani backendów.
- Każdy kod w `gulp-api` i `gulp-core` musi kompilować się i działać pod TeaVM. Zakazane w tych modułach: refleksja w czasie działania, `Class.forName`, dynamiczne proxy, `java.io.File`, `java.nio.file`, blokujące I/O, bezpośrednie tworzenie wątków, `System.exit`.
- Żadnych zależności zewnętrznych w `gulp-api`. W `gulp-core` tylko te wymienione w sekcji 5.
- Zero alokacji w gorących ścieżkach (pętla renderowania, dispatch eventów, fizyka): używaj pul obiektów i obiektów wielokrotnego użytku.

- Nie kopiuj kodu z Bukkita, Spigota, Papera, Minecrafta, libGDX ani Godota. Można czerpać idee, nie tekst źródłowy.
- Stan świata zmienia się tylko na głównym wątku, w ticku. Kod asynchroniczny wraca na główny wątek przez scheduler.
- Publiczne API jest niemutowalne w kontrakcie: po wydaniu 1.0 zmiany łamiące wymagają deprecjacji z jedną wersją przejściową.

Konwencje kodu

- Java 25, rekordy dla typów wartości, `sealed` dla zamkniętych hierarchii, pattern matching tam, gdzie poprawia czytelność.
- Pakiet bazowy `dev.gulp`. Nazwy klas, metod i komentarzy w kodzie po angielsku. Dokument jest po polsku.
- `Nullable` i `NotNull` z JSpecify na każdym publicznym parametrze i wartości zwracanej w API.
- Metody fluent zwracają `this` z typem generycznym tam, gdzie potrzebne (wzorzec `self()`).
- Jednostki: pozycje w jednostkach świata (1 jednostka = 1 kafelek domyślnie), czas logiki (scheduler, cooldowny, czas życia) w tickach lub `Duration`, czasy wizualne (tweeny, przejścia, wyciszanie dźwięku, efekty kamery) w sekundach jako `float`, kąty w stopniach w API (radiany wewnętrznie), prędkości w jednostkach świata na sekundę, kolory jako `Color` (RGBA float).
- Formatowanie: Spotless z palantir-java-format, uruchamiane w `check`.

Definicja ukończenia (dla każdego zadania)

- ☐ Publiczne API ma Javadoc z przykładem użycia.
- ☐ Testy jednostkowe przechodzą na backendzie headless, pokrycie logiki co najmniej 80%.
- ☐ Przykład w `examples/showcase` demonstruje funkcję i działa na desktopie i w przeglądarce.
- ☐ Brak nowych ostrzeżeń kompilatora i TeaVM.
- ☐ Zadanie odhaczone w roadmapie, wpis w `CHANGELOG.md`.

Pliki pomocnicze w repozytorium

- `CLAUDE.md` — skrót sekcji 1 plus polecenia build i test (tworzony w etapie 0).
- `docs/spec.md` — kopia tego dokumentu, aktualizowana przy zmianach.
- `docs/decisions/` — ADR-y dla decyzji podjętych w trakcie pracy.

2. Wizja i zakres

Gulp to framework gier 2D w Javie, który daje możliwości libGDX dla 2D, prostotę użycia Godota i styl architektoniczny serwera Minecrafta (Paper). Pisanie gry ma przypominać pisanie pluginu: klasa główna z cyklem życia, listenery eventów, zadania w schedulerze, rejestry z kluczami.

Cele

- **Parytet z libGDX dla 2D:** wszystko, co da się zrobić w 2D w libGDX, da się zrobić w Tesserze (pełna lista w sekcji 9).
- **Prostota z Godota:** UI bez ręcznego liczenia pozycji, ruch postaci jedną metodą, tweeny, akcje wejścia, skalowanie do rozdzielczości (sekcja 10).
- **Architektura w stylu Minecrafta:** encje + eventy + scheduler + rejestry, cienkie API oddzielone od implementacji.
- **Modularność:** gra dzieli się na moduły z jasnym „gdzie, co, jak i kiedy” — każdy moduł ma własny cykl życia, zależności, listenery, zadania i konfigurację.
- **Tylko kod:** sceny i interfejsy buduje się w Javie, deklaratywnym, czytelnym API. Bez edytora wizualnego i plików scen.
- **Platformy:** Windows, macOS, Linux (x64 i ARM64) oraz przeglądarki (Chrome, Firefox, Safari, Edge w aktualnych wersjach).
- **Nowoczesność:** Java 25, HiDPI, fonty MSDF, kształtowanie tekstu, hot reload zasobów, WebAssembly GC, gamepady z hot-plugiem, dostęp do narzędzi jak Aseprite i LDtk.

Poza zakresem

- Grafika 3D i fizyka 3D.
- Edytor wizualny, pliki scen, język skryptowy.
- Android i iOS (architektura ich nie blokuje, ale nie są celem 1.0).
- Multiplayer sieciowy w 1.0 (dostępne jest tylko HTTP i WebSocket jako narzędzia; pełny netcode to przyszły etap).
- Konsole do gier.

Dla kogo

Dla programistów Javy, którzy chcą pisać gry 2D kodem, bez edytora, a zwłaszcza dla osób znających Bukkita i Papera. Kod gry ma być zrozumiały bez znajomości OpenGL ani macierzy.

3. Architektura i słownik pojęć

Gra składa się z jednej klasy `Game` i dowolnej liczby klas `GameModule`, a silnik (`Engine`) pełni rolę serwera, który je uruchamia, tickuje i sprząta po nich. To ten sam układ co serwer Paper z pluginami, ale bez słowa „plugin” i z modułami jako jednostką podziału gry.

Zasady architektury

- Cienkie API, ukryta implementacja.** Kod gry widzi tylko interfejsy z `gulp-api`. Implementacja może się zmieniać bez psucia gier.
- Własność zasobów (ownership).** Wszystko, co moduł rejestruje (listenery, zadania, komendy, nakładki UI, dźwięki w odtwarzaniu), należy do niego i jest automatycznie usuwane przy jego wyłączeniu. Tak jak Bukkit anuluje zadania pluginu przy `onDisable`.
- Komunikacja przez eventy.** Moduły nie wołają się nawzajem bezpośrednio, tylko reagują na eventy lub korzystają z usług (`Services`). Dzięki temu moduł da się wyłączyć lub podmienić.
- Rejestry i klucze.** Typy treści (typy encji, kafelków, dźwięki, akcje wejścia, warstwy kolizji) są rejestrowane pod kluczem `namespace:id` w fazie `onLoad` i potem zamrażane.
- Tick jest jedynym miejscem zmian stanu.** Logika działa w stałym kroku, renderowanie tylko czyta stan i interpoluje.
- Dwie przestrzenie rysowania.** Świat (przez kamerę) i ekran (UI i nakładki, niezależne od kamery), jak `CanvasLayer` w Godocie.

Gdzie co jest, jak i kiedy

Co chcesz zrobić	Gdzie	Kiedy
Ustawić okno, rozdzielczość bazową, TPS	<code>Game.configure(GameSettings)</code>	przed startem platformy
Zarejestrować typy encji, kafelków, dźwięki, akcje	<code>onLoad()</code> modułu lub gry	przed załadowaniem zasobów i światów
Dodać listenery, zadania, komendy, UI	<code>onEnable()</code> modułu	po załadowaniu zasobów startowych
Zapisać stan, zwolnić zasoby	<code>onDisable()</code> modułu	przy wyłączeniu modułu lub gry
Reagować na zdarzenia	<code>@EventHandler</code> w <code>Listener</code>	w ticku, gdy event zostanie wywołany

Co chcesz zrobić	Gdzie	Kiedy
Zachowanie konkretnej encji	<code>Component</code> na typie encji	co tick, dla każdej encji z komponentem
Opóźnione lub cykliczne akcje	<code>scheduler()</code> modułu	w wybranym ticku
Menu, HUD	<code>ui()</code> i <code>Screen</code>	budowane w kodzie, odświeżane przez bindingi
Rysowanie własne	<code>RenderLayer</code> lub <code>RenderEvent</code>	w klatce renderowania

Cykl życia

```
sequenceDiagram
    participant L as Launcher
    participant E as Engine
    participant G as Game
    participant M as Moduły
    L->>E: start(new MyGame())
    E->>G: configure(settings)
    E->>E: init platformy (okno, GL, audio)
    E->>G: onLoad()
    E->>M: onLoad() w kolejności zależności
    E->>E: zamrożenie rejestrów, ładowanie zasobów startowych
    E->>G: onStart()
    E->>M: onEnable()
    E->>E: pętla gry
    E->>M: onDisable() w odwrotnej kolejności
    E->>G: onStop()
```

Kolejność modułów wynika z sortowania topologicznego zależności. Cykl zależności kończy start błędem z pełną ścieżką cyklu.

Fazy jednego ticku

1. Zebranie wejścia (stany klawiszy, eventy input).
2. `TickStartEvent`.
3. Zadania schedulera zaplanowane na ten tick.
4. Tick światów: komponenty encji, zachowania, kafelki z tickiem.
5. Fizyka i kolizje, eventy kolizji i triggerów.
6. Tweeny i animacje logiczne.

7. Usunięcie encji oznaczonych do usunięcia, `TickEndEvent` .

Renderowanie działa osobno, tyle razy na sekundę, ile pozwala VSync, z interpolacją pozycji między dwoma ostatnimi tickami.

Wejście dla UI (klik, hover, pisanie) jest obsługiwane w każdej klatce, żeby interfejs reagował natychmiast; akcje gameplay są odczytywane w ticku. Encje zespawnowane w trakcie ticku zaczynają tickować od następnego ticku, a usunięte znikają na jego końcu.

Słownik pojęć

Pojęcie	Znaczenie	Odpowiednik (Paper / Godot / libGDX)
<code>Engine</code>	Silnik, dostęp do wszystkich usług	<code>Server</code> / <code>Engine</code> / <code>Gdx</code>
<code>Gulp</code>	Statyczna fasada: <code>Gulp.engine()</code>	<code>Bukkit</code> / — / <code>Gdx</code>
<code>Game</code>	Klasa główna gry z cyklem życia	główna klasa pluginu / — / <code>ApplicationListener</code>
<code>GameModule</code>	Wydzielona część gry z własnym cyklem życia	plugin / autoload / —
<code>Listener</code> , <code>@EventHandler</code>	Obsługa eventów	to samo / sygnały / —
<code>Scheduler</code> , <code>Task</code>	Zadania opóźnione i cykliczne	<code>BukkitScheduler</code> / <code>Timer</code> , <code>await</code> / <code>Timer</code>
<code>World</code>	Poziom gry: encje, mapa, kamera	<code>World</code> / scena / —
<code>Entity</code> , <code>EntityType</code>	Obiekt w świecie i jego typ	to samo / węzeł / —
<code>Component</code>	Zachowanie lub dane podłączone do encji	AI goal, PDC / węzeł dziecko / —
<code>TileMap</code> , <code>TileType</code>	Mapa kafelkowa i typ kafelka	blok, <code>Material</code> / <code>TileMapLayer</code> / <code>TiledMap</code>
<code>Key</code> , <code>Registry</code>	Identyfikator <code>namespace:id</code> i rejestr typów	<code>NamespacedKey</code> , <code>Registry</code> / — / —
<code>DataContainer</code>	Dowolne dane na encji, świecie, kafelku	<code>PersistentDataContainer</code> / metadane / —

Pojęcie	Znaczenie	Odpowiednik (Paper / Godot / libGDX)
<code>Services</code>	Rejestr usług między modułami	<code>ServicesManager</code> / autoload / —
<code>Screen</code>	Pełnoekranowy stan UI (menu, pauza)	GUI ekwipunku / scena UI / <code>Screen</code>
<code>Overlay</code>	Rysowanie po ekranie w trybie natychmiastowym	— / <code>CanvasLayer</code> + <code>_draw</code> / <code>SpriteBatch</code> w HUD

4. Kwestie prawne

Czerpanie idei architektonicznych z Bukkita, Godota i libGDX jest bezpieczne, o ile cały kod i zasoby powstają od zera. To nie jest porada prawna; przed komercyjnym wydaniem warto skonsultować się z prawnikiem.

- **Kod.** Nie kopiować kodu Bukkita i CraftBukkita (GPL — skopiowanie wymusiłoby GPL na całym frameworku), Minecrafta (własnościowy), libGDX (Apache 2.0) ani Godota (MIT). Apache i MIT pozwalają na kopiowanie z zachowaniem informacji o licencji, ale dla czystości projektu piszemy wszystko samodzielnie.
- **Nazwy.** Własne pakiety (`dev.gulp`). Ogólne pojęcia (`Listener` , `EventPriority` , `Scheduler` , `Registry` , `Tween`) są powszechne. Unikamy nazw markowych: `Bukkit` , `Craft*` , `Minecraft` , `Godot` .
- **Znaki towarowe.** „Minecraft”, „Bukkit”, „Godot” nie pojawiają się w nazwie ani logo. W opisie można napisać, że API jest inspirowane stylem pluginów serwerowych. Przed publikacją sprawdzić nazwę „Gulp” pod kątem znaków towarowych i pomyłek z narzędziem JavaScript `gulp.js`.
- **Zasoby.** Wszystkie tekstury, dźwięki i fonty w przykładach są własne lub na licencji CC0 (np. Kenney). Font domyślny: licencja OFL.
- **Zależności.** Tylko licencje zgodne z Apache 2.0 (BSD, MIT, Apache, zlib, OFL dla fontów). Lista i licencje w `THIRD_PARTY_NOTICES.md` , generowana w buildzie.
- **Licencja frameworka.** Proponowana: Apache 2.0 (gry mogą być zamknięte, jest klauzula patentowa). Decyzja w sekcji 23.

5. Stos technologiczny i wersje

Bazowa wersja to Java 25, bo TeaVM obsługuje ją od wydania 0.13, a aktualne TeaVM 0.15 i LWJGL 3.4.3 są aktywnie rozwijane. Wersje poniżej są punktem startowym; Claude aktualizuje je w etapie 0 do najnowszych stabilnych.

Element	Wersja / wybór	Uwagi
Język	Java 25 (LTS)	Rekordy, <code>sealed</code> , pattern matching. TeaVM wspiera Java 25 od 0.13 (źródło)
Build	Gradle 9.x, Kotlin DSL, version catalog <code>libs.versions.toml</code>	Convention plugins w <code>build-logic/</code>
Desktop: okno i input	LWJGL 3.4.3: GLFW 3.5.1	(źródło)
Desktop: grafika	LWJGL OpenGL, profil 3.3 core	Działa na macOS (max 4.1 core), Windows, Linux
Desktop: audio	LWJGL OpenAL Soft	Mikser własny w Javie nad źródłami OpenAL
Desktop: obrazy, OGG	LWJGL stb_image, stb_vorbis	
Desktop: tekst	LWJGL FreeType + HarfBuzz	Kształtowanie tekstu, fallback fontów
Narzędzia: fonty MSDF	LWJGL msdfgen	Tylko w <code>gulp-tools</code> i podczas buildu
Web: kompilator	TeaVM 0.15.x, cel Wasm GC, fallback JS	Wasm GC ma korutyny (emulacja <code>Thread</code>) od 0.13; aktualne 0.15.0 wymaga JDK 17+ do uruchomienia kompilatora (źródło)
Web: grafika / audio / input	WebGL2, WebAudio, DOM + Gamepad API	Przez JSO (interop TeaVM)
Web: zapis danych	IndexedDB, fallback localStorage	
Formaty danych	Własny parser JSON i podzbioru YAML w <code>gulp-core</code>	Muszą działać pod TeaVM, bez refleksji
Adnotacje null	JSpecify 1.0	
Testy	JUnit Jupiter, AssertJ, backend headless	Testy wizualne: porównanie zrzutów (sekcja 20)
Formatowanie	Spotless + palantir-java-format	

Element	Wersja / wybór	Uwagi
Importy narzędzi	Tiled (<code>.tmj</code>), LDtk (<code>.ldtk</code>), Aseprite (eksport JSON), TexturePacker-kompatybilny atlas JSON	Wszystkie formaty JSON, parsowane własnym parserem

Zasada: `gulp-api` nie ma zależności poza JSpecify. `gulp-core` nie ma zależności zewnętrznych. Zależności natywne żyją wyłącznie w backendach i narzędziach.

6. Moduły Gradle i układ pakietów

Repozytorium to jeden build Gradle z 11 modułami, w którym kod gry zależy tylko od `gulp-api`, a backend wybiera launcher. Zależności: `api` ← `core` ← `backend-*`; `platform` jest widoczne tylko dla `core` i backendów.

Moduł	Rola	Zależy od
<code>build-logic</code>	Convention plugins: wersja Javy, Spotless, testy, publikacja	—
<code>gulp-api</code>	Interfejsy, eventy, typy wartości, adnotacje, statyczna fasada <code>Gulp</code>	JSpecify
<code>gulp-platform</code>	SPI platformy: grafika, audio, input, pliki, okno, czas, wątki	—
<code>gulp-core</code>	Implementacja API: pętla, eventy, scheduler, światy, encje, fizyka, rendering, UI, zasoby	<code>api</code> , <code>platform</code>
<code>gulp-backend-desktop</code>	LWJGL3: GLFW, OpenGL 3.3, OpenAL, FreeType/HarfBuzz	<code>core</code>
<code>gulp-backend-web</code>	TeaVM: WebGL2, WebAudio, DOM, IndexedDB	<code>core</code>
<code>gulp-backend-headless</code>	Bez okna i dźwięku, grafika jako no-op lub software; do testów i CI	<code>core</code>
<code>gulp-processor</code>	Procesor adnotacji: dispatch eventów, codeki komponentów, walidacja modułów	<code>api</code> (w czasie kompilacji)
<code>gulp-test</code>	Narzędzia testowe dla gier: <code>GameTestHarness</code> , symulacja input, tickowanie, rzuty	<code>core</code> , <code>backend-headless</code>

Moduł	Rola	Zależy od
gulp-tools	CLI: packer atlasów, generator fontów MSDF i bitmapowych, importery	core, LWJGL (msdfgen, stb)
gulp-gradle-plugin	Plugin Gradle dla projektów gier (sekcja 7)	tools

Przykłady w `examples/`: `showcase` (każda funkcja na osobnym ekranie), `platformer`, `topdown` (proceduralny świat z chunkami), `ui-gallery` (wszystkie widgety).

Pakiety gulp-api

```
dev.gulp.api
├─ Gulp, Engine, Game, GameSettings, Platform (informacje read-only)
├─ module      GameModule, ModuleInfo, ModuleManager, ModuleState
├─ event       Event, Cancellable, Listener, EventHandler, EventPriority,
HandlerList, Events
|   └─ lifecycle / world / entity / physics / input / ui / asset
├─ scheduler   Scheduler, Task, TaskRunnable, Sequence
├─ registry    Key, Keyed, Registry, Registries, RegistryKey
├─ data        DataContainer, DataType, Json, JsonValue, Config,
ConfigSection, Codec
├─ math        Vec2, MutableVec2, Rect, Circle, Polygon, Transform2D, Mat3,
Mathf, Ease,
|               Interpolation, Rng, Noise, Bezier, CatmullRom, Intersect,
Grid
├─ graphics    Color, Texture, TextureRegion, NinePatch, Pixmap, Shader,
Material,
|               BlendMode, FrameBuffer, TextureFilter, TextureWrap
├─ render      Draw (API rysowania), RenderLayer, Camera, Display,
StretchMode, Light, Occluder
├─ text        Font, Text, TextStyle, Markup, TextAlign
├─ asset       Assets, Asset, AssetKey, AssetType, LoadingProgress,
ResourcePack
├─ world       World, Worlds, Location, Chunk, ChunkGenerator, TileMap,
TileLayer,
|               Tile, TileType, TileSet, Terrain, Parallax
├─ entity      Entity, EntityType, Component, Tags, EntityQuery
|   └─ component  gotowe komponenty (sekcja 14)
├─ physics     Physics, Body, BodyType, Shape, CollisionLayer, Contact,
RayHit, Mover
├─ nav         NavGrid, Pathfinder, Path, NavAgent
├─ ai          StateMachine, BehaviorTree, Steering
├─ input       Input, InputAction, Binding, Keys, KeyCode, Mouse, Gamepad,
TextInput
```

└─ audio	Audio, Sound, Music, Bus, Playback
└─ anim	Tween, Tweens, SpriteAnimation, Animator, Timeline, Property
└─ particle	ParticleEffect, EmitterConfig, Particles
└─ ui	Ui, Screen, Node, containers, widgets, Theme, Style, Anchor, Size, Insets,
	State, Binding, Overlay
└─ command	Command, CommandContext, Arguments, Console
└─ service	Services
└─ save	SaveStore, SaveSlot, Preferences
└─ i18n	Translations, LocaleInfo
└─ net	Http, HttpRequest, HttpResponse, WebSocket
└─ debug	Debug, DebugDraw, Profiler, Stats

Pakiety `gulp-core` odzwierciedlają to drzewo pod `dev.gulp.core` z klasami `Gulp*Impl` lub nazwami opisowymi (np. `EventBusImpl`, `SpriteBatcher`). Pakiet `internal` w `core` nie jest eksportowany (JPMS `module-info.java` na desktopie).

7. Warstwa platformy: desktop, web, headless

Warstwa platformy to zestaw interfejsów SPI w `gulp-platform`, które każdy backend implementuje w całości; `gulp-core` nigdy nie wie, na jakiej platformie działa. Wspólnym mianownikiem grafiki jest OpenGL ES 3.0 / WebGL2.

Interfejsy SPI

Interfejs	Odpowiedzialność	Desktop	Web	Headless
<code>PlatformLoop</code>	Kto napędza klatki: wywołuje <code>engine.frame(nanoTime)</code>	pętla <code>while</code> na głównym wątku	<code>requestAnimationFrame</code>	ręczne <code>step(n)</code> w testach
<code>GL</code>	Cienka abstrakcja GLES3: bufor, VAO, shadery, tekstury, FBO, stan blendingu	LWJGL GL33	WebGL2 przez JSO	no-op z licznikiem wywołań
<code>PlatformWindow</code>	Rozmiar, rozmiar framebuffera, skala DPI, tytuł, ikona, tryby okna, VSync, cursor, focus, zamknięcie	GLFW	canvas, Fullscreen API, <code>devicePixelRatio</code>	stały rozmiar
<code>PlatformInput</code>	Klawisze (keycode + scancode), tekst, mysz, scroll, dotyk, gamepady, schowek	GLFW + mapowania gamepadów SDL	DOM, Pointer Events, Gamepad API	kolejka zdarzeń wstrzykiwana w testach
<code>PlatformAudio</code>	Głosy (odtwarzanie PCM), strumienie, głośność, pitch, panorama	OpenAL Soft	WebAudio	cisza z licznikami
<code>PlatformFiles</code>	Asynchroniczny odczyt zasobów, zapis danych użytkownika, manifest zasobów	pliki + katalog danych aplikacji systemu	<code>fetch</code> + IndexedDB	pamięć RAM

Interfejs	Odpowiedzialność	Desktop	Web	Headless
PlatformDecoders	Obrazy (PNG, JPG, WebP), audio (OGG, WAV), rasteryzacja glików	stb_image, stb_vorbis, FreeType + HarfBuzz	createImageBitmap, decodeAudioData, Canvas2D	stuby
PlatformExecutor	Zadania asynchroniczne poza głównym tickiem	wątki wirtualne	kolejka kooperacyjna między klatkami	synchronicznie
PlatformNet	HTTP i WebSocket	java.net.http	fetch, WebSocket	atrapy
PlatformInfo	System, czy web, język systemu, rozmiar ekranu, GPU, możliwości			
PlatformModules	Kod generowany przez procesor: punkt wejścia, dispatchery eventów, codeki komponentów	rejestr generowany przez procesor	ten sam rejestr	ten sam rejestr

Wszystkie operacje asynchroniczne zwracają własny typ `Promise<T>` z `gulp-core` (nie `CompletableFuture`), który zawsze woła callbacki na głównym wątku gry.

Shadery

Shadery pisze się w GLSL ES 3.00. Backend desktop podmienia nagłówki `#version 300 es` na `#version 330 core` i usuwa kwalifikatory precyzji. Silnik dostarcza bibliotekę include'ów (`#include "gulp:common.glsl"`) rozwijaną w czasie ładowania.

Desktop

- Na macOS GLFW wymaga głównego wątku: launcher sprawdza to i przy braku `-XstartOnFirstThread` restartuje JVM z tą flagą.
- Katalog danych: `%APPDATA%/<gra>` (Windows), `~/Library/Application Support/<gra>` (macOS), `$XDG_DATA_HOME/<gra>` (Linux).
- HiDPI: logika i UI w punktach logicznych, framebuffer w pikselach fizycznych.
- Pakowanie: `jlink` (własne środowisko uruchomieniowe) + `jpackage` (MSI/EXE, DMG/APP, DEB/ApplImage).

Web

- Na webie nie ma dynamicznego ładowania klas: punkt wejścia generuje plugin Gradle z `mainClass`, a moduły są podawane jawnie w `GameSettings.modules(...)`, więc TeaVM widzi cały kod w czasie kompilacji.
- Refleksja jest niedostępna, więc dispatch eventów i serializacja używają kodu generowanego.
- Nie da się blokująco czekać na plik. Ładowanie zasobów jest asynchroniczne, z ekranem ładowania.

- Nie da się listować katalogów. Build generuje `assets.manifest.json` z listą plików i rozmiarów.
- Audio i pełny ekran wymagają gestu użytkownika. Backend pokazuje ekran „Kliknij, aby zacząć”, jeśli gra chce dźwięk od startu.
- Przy ukryciu karty (Page Visibility API) gra dostaje `FocusLostEvent`, a pętla zwalnia. Zamknięcie karty nie gwarantuje wywołania `onDisable` ani `onStop`, dlatego stan zapisuje się przy ukryciu karty (sekcja 19.1).
- Klawisze gry blokują domyślne akcje przeglądarki (strzałki, spacja, Tab) tylko, gdy canvas ma focus.
- Strona HTML z własnym ekranem ładowania (CSS) pokazuje postęp pobierania Wasm, zanim Java wystartuje.
- Cel domyślny: Wasm GC. Fallback JS dla przeglądarek bez Wasm GC, wybierany automatycznie przez loader.

Headless

Backend bez okna do testów jednostkowych i integracyjnych, CI oraz symulacji. Pozwala tickować grę szybciej niż w czasie rzeczywistym i wstrzykiwać wejście.

Plugin Gradle dla projektów gier

```
plugins { id("dev.gulp.game") version "0.1.0" }

gulp {
    mainClass = "com.example.coins.CoinGame"
    title = "Coin Hunter"
    desktop { icon = file("icon.png") }
    web { target = WebTarget.WASM_GC; jsFallback = true }
    assets {
        atlas("sprites", "assets/sprites")
        msdfFont("ui", "assets/fonts/Inter.ttf", charset =
Charset.LATIN_EXTENDED)
    }
}
```

Zadanie	Działanie
<code>runDesktop</code>	Uruchamia grę z hot reloadem zasobów
<code>runWeb</code>	Serwer deweloperski na <code>localhost:8080</code> z automatycznym odświeżaniem
<code>buildWeb</code> / <code>packageWeb</code>	Statyczna strona / ZIP gotowy dla itch.io

Zadanie	Działanie
<code>packageDesktop</code>	Instalatory dla bieżącego systemu (jlink + jpackage)
<code>packAssets</code>	Pakowanie atlasów, generowanie fontów, manifest zasobów
<code>generateAssetKeys</code>	Klasa ze stałymi dla każdego zasobu (<code>GameAssets.Textures.PLAYER</code>), jak klasa <code>R</code> w Androidzie
<code>checkApiUsage</code>	Pada, jeśli kod gry importuje coś spoza <code>dev.gulp.api</code>

8. Rdzeń

Rdzeń to siedem elementów, z których korzysta każdy inny podsystem: `Engine`, `Game`, `GameModule`, eventy, scheduler, rejestry z kluczami oraz konfiguracja z danymi. Buduje się go w etapie 1, przed jakąkolwiek grafiką.

8.1 Engine i fasada Gulp

`Gulp.engine()` zwraca jedyną instancję `Engine`. Metody dostępne: `game()`, `modules()`, `events()`, `scheduler()`, `registries()`, `assets()`, `worlds()`, `input()`, `audio()`, `ui()`, `display()`, `services()`, `saves()`, `preferences()`, `translations()`, `commands()`, `http()`, `debug()`, `platform()`, `logger()`.

Stan i czas: `tick()` (licznik ticków), `tps()` (zmierzony), `targetTps()`, `timeScale()` / `setTimeScale(float)` (spowolnienie i przyspieszenie), `isPaused()` / `pause()` / `resume()`, `stop()`.

8.2 Game i GameSettings

```
public abstract class Game {
    public void configure(GameSettings settings) {} // okno, rozdzielczość,
    TPS; przed startem platformy
    public void onLoad() {} // rejestracje w
    rejestrach
    public abstract void onStart(); // pierwszy świat lub
    ekran
    public void onStop() {}
    public final Engine engine() { ... }
    public final Logger logger() { ... }
    public final Config config() { ... } // config/game.yml
    public final Key key(String path) { ... } // Key.of(id(), path)
    public abstract String id(); // namespace, np.
    "coins"
}
```

`GameSettings` (budowniczy): `title`, `windowSize(w, h)`, `resizable`, `fullscreen`, `vsync`, `targetFps` (0 = bez limitu), `ticksPerSecond` (domyślnie 60), `baseResolution(w, h)`, `stretchMode`, `aspectMode`, `pixelPerfect`, `clearColor`, `icon`, `modules(GameModule...)`.

8.3 GameModule

Moduł to wydzielona część gry (walka, ekwipunek, dialogi, HUD, zapis). Każdy ma własny cykl życia i jest właścicielem wszystkiego, co zarejestruje.

```
@ModuleInfo(id = "combat", dependsOn = {"world"}, softDependsOn =
{"audio_fx"})
public final class CombatModule extends GameModule {
    @Override public void onLoad() { /* rejestracja typów */ }
    @Override public void onEnable() { listen(new DamageListener());
    every(20, this::regen); }
    @Override public void onDisable() { /* zapis, sprzątanie własne */ }
}
```

- Metody pomocnicze klasy bazowej: `listen(listener)`, `on(EventClass, handler)`, `later(ticks, task)`, `every(ticks, task)`, `command(name, executor)`, `config()` (plik `config/<id>.yaml`), `data()` (`DataContainer` modułu zapisywany z grą), `logger()`, `key(path)`, `require(OtherModule.class)`.
- Skróty do usług silnika w `GameModule` i `Game`: `registries()`, `worlds()`, `ui()`, `input()`, `audio()`, `assets()`, `tr(...)` i pozostałe z 8.1. `Component` ma skróty `input()`, `ui()` i `audio()`.

- `ModuleManager`: `get(Class)`, `isEnabled(id)`, `enable(id)`, `disable(id)` w trakcie gry. Wyłączenie modułu wyłącza najpierw moduły od niego zależne.
- `ModuleState`: `LOADED`, `ENABLED`, `DISABLED`, `FAILED`. Wyjątek w `onEnable` przestawia moduł w `FAILED`, loguje błąd i wyłącza moduły zależne; gra działa dalej.
- Moduły deklaruje się w `GameSettings.modules(...)`. Procesor adnotacji sprawdza w czasie kompilacji unikalność `id` i zależności między modułami projektu; pełna walidacja (także modułów z bibliotek) odbywa się przy starcie. `onLoad` wykonuje się przy starcie dla wszystkich zadeklarowanych modułów, także tych z `@ModuleInfo(enabledByDefault = false)`, bo potem rejestry są zamrażane; późniejsze włączenie takiego modułu wywołuje tylko `onEnable`.
- Mody zewnętrzne (JAR-y z modułami ładowane z katalogu `mods/`) są tylko na desktopie i dopiero po 1.0.

8.4 Eventy

- `Event` (klasa abstrakcyjna), `Cancellable` (`isCancelled`, `setCancelled`), bez boilerplate'u znanego z Bukkita: procesor generuje listę handlerów dla każdej klasy eventu, a `Events.hasListeners(EventClass)` daje szybką ścieżkę, gdy nikt nie słucha.
- `EventPriority`: `LOWEST`, `LOW`, `NORMAL`, `HIGH`, `HIGHEST`, `MONITOR` (tylko odczyt). `@EventHandler(priority, ignoreCancelled)`.
- Rejestracja: `listen(Listener)` z adnotacjami albo lambda `on(EntityDamageEvent.class, EventPriority.HIGH, e -> ...)` zwracająca `Subscription` z `cancel()`.
- Eventy przypięte do obiektu: `entity.on(EntityDamageEvent.class, e -> ...)` słucha tylko eventów tej encji i znika razem z nią (odpowiednik sygnałów w Godocie). Analogicznie `widget.on(...)` w UI.
- Wywołanie: `events().call(event)` zwraca event, by sprawdzić `isCancelled()`.
- Implementacja: procesor adnotacji generuje dla każdego `Listener` klasę `<Nazwa>$Handlers` z bezpośrednią obsługą każdej metody. Tablica handlerów per typ eventu jest przebudowywana tylko przy zmianie rejestracji. Dziedziczenie eventów: listener na klasie bazowej dostaje eventy podklas.
- Wyjątek w handlerze jest logowany z nazwą modułu i nie przerywa pozostałych handlerów. Rejestracje zmienione w trakcie wywołania eventu działają od następnego wywołania; wywołanie eventu wewnątrz handlera jest dozwolone do głębokości 64, potem błąd z łańcuchem eventów.
- Zasada: eventy tylko dla zdarzeń dyskretnych. Nie ma eventu „encja się poruszyła” co tick; do ciągłej logiki służą komponenty.

- Katalog eventów jest w sekcjach podsystemów; eventy cyklu życia: `GameStartEvent`, `GameStopEvent`, `ModuleEnableEvent`, `ModuleDisableEvent`, `TickStartEvent`, `TickEndEvent`, `PauseEvent`, `ResumeEvent`, `FocusGainedEvent`, `FocusLostEvent`, `WindowResizeEvent`.

8.5 Scheduler

- `run(task)` (następny tick), `later(delay, task)`, `every(period, task)`, `every(delay, period, task)`; `delay` i `period` w tickach (`long`) lub `Duration`. Zwracają `Task` z `cancel()`, `isCancelled()`, `owner()`.
- `TaskRunnable` (odpowiednik `BukkitRunnable`): klasa z `run()` i `cancel()` wewnątrz, plus `runEvery(owner, period)`.
- `Sequence` zamiast `await` z Godota, bez ręcznego liczenia ticków:

```
scheduler().sequence()
    .run(() -> door.open())
    .wait(Duration.ofSeconds(1))
    .waitUntil(() -> player.isOnFloor())
    .run(() -> camera.shake(0.3f, 0.2f))
    .repeat(3)
    .start(this);
```

- Asynchroniczność: `async(supplier).thenSync(consumer)` — praca poza głównym tickiem, wynik z powrotem na głównym wątku. Na webie „async” oznacza „pomiędzy klatkami”, nie równoległe; Javadoc musi to mówić.
- Zadania są domyślnie wstrzymywane podczas pauzy i skalowane przez `timeScale`. `realtime()` tworzy zadania niezależne od pauzy (np. animacje menu pauzy).
- Zadania należą do właściciela (`Game` lub `GameModule`) i są anulowane przy jego wyłączeniu.

8.6 Rejestry i klucze

- `Key`: `namespace:path`, `namespace` `[a-z0-9_.-]+`, `path` `[a-z0-9_./-]+`. Tworzenie: `Key.of("coins", "coin")`, `key("coin")` w module, `Key.parse("coins:coin")`. Namespace `gulp` zarezerwowany.
- `Registry<T extends Keyed>`: `register(key, value)`, `get(key)`, `getOrThrow`, `keys()`, `values()`, `stream()`, `contains`. Rejestracja tylko w `onLoad`; potem rejestr jest zamrożony, a próba zapisu rzuca `IllegalStateException`.
- Rejestry wbudowane (`Registries`): `ENTITY_TYPE`, `TILE_TYPE`, `COMPONENT_TYPE`, `SOUND`, `PARTICLE_EFFECT`, `INPUT_ACTION`, `COLLISION_LAYER`, `DAMAGE_TYPE`, `TRANSITION`, `THEME`. Gra może tworzyć własne:

```
registries().create(key("items"), Item.class).
```

8.7 Usługi

`services().register(Class<T>, provider, owner, ServicePriority)`, `services().get(Class<T>)` (najwyższy priorytet), `getAll`. Usługa znika przy wyłączeniu właściciela i wywołuje `ServiceRegisterEvent` / `ServiceUnregisterEvent`.

8.8 Konfiguracja i dane

- `Config` w formacie YAML (podzbiór: mapy, listy, skalary, komentarze, wielolinijkowe stringi). Pliki: `config/game.yml` i `config/<moduł>.yml`, wartości domyślne z zasobów (`assets/<id>/config/...`).
- Odczyt: `getString(path, def)`, `getInt`, `getFloat`, `getBoolean`, `getList`, `getSection`, `get(path, Codec<T>)`. Zapis: `set`, `save()`. `reload()` wywołuje `ConfigReloadEvent`.
- `Codec<T>`: dwukierunkowa konwersja typów do `JsonValue`. Dla rekordów oznaczonych `@Serializable` procesor generuje codec automatycznie (bez refleksji).
- `DataContainer`: `set(Key, DataType<T>, value)`, `get`, `getOrDefault`, `has`, `remove`, `keys()`. Typy wbudowane: `BOOLEAN`, `INT`, `LONG`, `FLOAT`, `DOUBLE`, `STRING`, `KEY`, `VEC2`, `LIST(t)`, `MAP(t)`, `CONTAINER`, `of(Codec)`. Dostępny na `Entity`, `World`, `Tile` (dla kafelków ze stanem), `GameModule`, `SaveSlot`.

8.9 Komendy i konsola

- `commands().register(Command)` z budowniczym: nazwa, aliasy, opis, argumenty typowane (`int`, `float`, `string`, `key`, `entityType`, `enum`), podkomendy, podpowiedzi. Wykonanie dostaje `CommandContext` z `arg("name")` i `reply(Text)`.
- Konsola deweloperska pod `~` (tylko w buildzie deweloperskim, włączana w `GameSettings`), z historią i autouzupełnianiem; na desktopie także w terminalu.
- Komendy wbudowane: `/help`, `/tps`, `/modules`, `/module enable|disable <id>`, `/reload config|assets`, `/timescale <x>`, `/spawn <typ>`, `/tp <x> <y>`, `/debug <flag>`, `/profile start|stop`.

9. Parytet z libGDX dla 2D

Każda funkcja 2D z libGDX i jego oficjalnych rozszerzeń ma odpowiednik w Tessera; wersja 1.0 nie wychodzi, dopóki każdy wiersz tej tabeli nie jest zrealizowany. Kolumna „Etap” odnosi się do roadmapy w sekcji 22.

Obszar	libGDX	Gulp	Sekcja	Etap
Cykl życia	ApplicationListener, Game, Screen	Game, GameModule, Screen UI, World	3, 8	1
Rysowanie sprite'ów	SpriteBatch, PolygonSpriteBatch	Draw z automatycznym batchingiem, wielokąty z teksturami	12	2
Tekstury	Texture, TextureRegion, TextureAtlas, Sprite, NinePatch	Texture, TextureRegion, atlasy z packera, SpriteComponent, NinePatch	12, 13	2, 4
Obraz CPU	Pixmap	Pixmap: rysowanie pikseli, kształtów, kopiowanie, zapis PNG	12	2
Kształty	ShapeRenderer	Draw.rect/circle/line/polygon/arc wypełnione i obrysy, z antyaliasingiem	12	2
Bufory i shadery	FrameBuffer, ShaderProgram, Mesh	FrameBuffer, Shader, Material, Mesh2D	12	2
Kamera i viewпорты	OrthographicCamera, Fit/Fill/Stretch/Extend/ScreenViewport	Camera, Display z StretchMode i AspectMode	12	2
Przycinanie	ScissorStack	Draw.clip(rect, () -> ...)	12	2
Zrzut ekranu	ScreenUtils	display().screenshot() → Pixmap	12	2
Fonty	BitmapFont, FreeType, fonty distance field, GlyphLayout	Fonty bitmapowe, MSDF, rasteryzacja w locie, TextLayout, markup	13	4
Animacja klatkowa	Animation<TextureRegion>	SpriteAnimation, import z Aseprite	17	8
Interpolacje	Interpolation	Ease (pełen zestaw krzywych) + Tween	11, 17	2, 8
Cząsteczki	ParticleEffect, edytor cząsteczek	ParticleEffect definiowany w kodzie lub JSON	17	8
Scene graph UI	Stage, Actor, Group, Actions	Drzewo UI z Node, tweeny na właściwościach	18	9
Layout UI	Table, Container, Stack, HorizontalGroup, VerticalGroup	Kontenery w stylu Godota: Row, Column, Grid, Stack, Center, Margin, Flow, Split, Scroll	18	9
Widgety	Label, TextButton, ImageButton, CheckBox, SelectBox, List, ScrollPane, Slider, ProgressBar, TextField, TextArea, Tree, Window, Dialog, SplitPane, Touchpad, Tooltip, DragAndDrop	Wszystkie + Tabs, Foldable, RichText, SpinBox, ColorPicker, ItemGrid	18	9
Motywy UI	Skin (JSON)	Theme w kodzie, dziedziczony w drzewie, warianty stylów	18	9
Input	InputProcessor, InputMultiplexer, GestureDetector	Eventy input, akcje, warstwy obsługi (UI przed światem), gesty	16	5
Kontrolery	rozszerzenie.gdx-controllers	Wbudowane gamepady z hot-plugiem i wibracjami	16	5
Audio	Sound, Music, AudioDevice, AudioRecorder	Sound, Music, szyny głośności, dźwięk pozycyjny, surowe PCM; nagrywanie tylko desktop	16	5
Pliki	FileHandle (internal/local/external), Preferences	Assets (tylko odczyt), SaveStore, Preferences	13, 19	3, 10

Obszar	libGDX	Gulp	Sekcja	Etap
Mapy	TiledMap, loadery TMX, renderery ortogonalne, izometryczne, heksagonalne, animowane kafelki, obiekty map	TileMap z chunkami, import Tiled i LDtk, orientacje ortogonalna, izometryczna, heksagonalna, animowane kafelki, autotiling	14	6
Fizyka	rozszerzenie Box2D, Box2DLights	Kinematyka w stylu Godota + pełna fizyka brył sztywnych w czystej Javie; światło 2D z cieniami	15, 12	7, 8
Matematyka	Vector2, Matrix3, MathUtils, Rectangle, Circle, Polygon, Intersector, Bezier, CatmullRom, BSpline, RandomXS128, triangulacja, otoczka wypukła	Wszystkie + szum (Perlin, Simplex), Grid	11	2
AI	rozszerzenie.gdx-ai:steering, A*, drzewa zachowań, maszyny stanów, wiadomości	NavGrid, A*, NavAgent, steering, StateMachine, BehaviorTree; wiadomości to eventy	15	7
ECS	rozszerzenie Ashley	Encje + komponenty + zapytania (EntityQuery)	14	6
Zasoby	AssetManager (async, liczenie referencji, zależności)	Assets z tym samym zakresem + hot reload + resource packi	13	3, 4
Narzędzia	TexturePacker, Hiero, 2D Particle Editor	packAssets w Gradle, generator MSDF; cząsteczki w kodzie z podglądem hot reload	7, 13	4, 11
Serializacja	Json, XmlReader	Json + Codec generowane dla rekordów; XML poza zakresem	8	1
Lokalizacja	I18NBundle	Translations z formatowaniem i liczbą mnogą	19	10
Narzędzia ogólne	Pool, kolekcje prymitywne, Timer, Logger, Disposable, PerformanceCounter, GLProfiler	Pool, IntList / IntMap w core, scheduler, logger, automatyczne zwalnianie przez własność, profiler, statystyki GL	8, 20	1, 11
Sieć	Net: HTTP, gniazda TCP, openURI	HTTP, WebSocket, openUr1; TCP poza zakresem (web go nie ma)	19	10
System	schowek, kursory, tryby wyświetlania, monitory	Schowek, kursory systemowe i własne, pełny ekran, okno bez ramki, wybór monitora (desktop)	7, 16	0, 5

Skróty względem libGDX, które Gulp dodaje ponad parytet: UI bez ręcznego liczenia pozycji, ruch postaci jedną metodą, tweeny na dowolnych właściwościach, akcje wejścia z osiami i wektorami, skalowanie do rozdzielczości bazowej jednym ustawieniem. Szczegóły w sekcji 10.

10. Uproszczenia przejęte z Godota

Gulp przejmuje z Godota 4 te mechanizmy, które usuwają ręczną matematykę i powtarzalny kod, ale w formie kodu Java zamiast edytora i węzłów. Najważniejsze jest UI oparte na kontenerach: w Godocie dziecko kontenera oddaje mu kontrolę nad pozycją, a o rozmiarze decydują flagi Fill, Expand, Shrink i Stretch Ratio ([dokumentacja Godota](#)).

Godot	Co upraszcza	Gulp	Sekcja
Kontenery (HBox , VBox , Grid , Margin , Center , Panel , Scroll , Split , Flow , Tab)	Układanie UI bez liczenia współrzędnych	Row , Column , Grid , Margin , Center , Panel , Scroll , Split , Flow , Tabs	18
Flagi rozmiaru: Fill, Expand, Shrink Begin/Center/End, Stretch Ratio	Podział wolnego miejsca bez obliczeń	.fill() , .expand() , .expand(2) , .shrinkCenter() na każdym węźle UI	18
Kotwice i presety (Full Rect, Center, Top Right itd.)	Przyklejanie elementów do krawędzi ekranu przy dowolnej rozdzielczości	.anchor(Anchor.TOP_RIGHT).offset(8, 8) , .fillParent()	18
FoldableContainer (Godot 4.5)	Rozwijane sekcje (akordeon) z grupowaniem (źródło)	Foldable z FoldGroup	18
Motywy z wariantami typów i nadpisaniami	Jednolity wygląd, zmiana stylu w jednym miejscu	Theme dziedziczony w drzewie, .variant("danger") , .style(s -> ...)	18
Automatyczna nawigacja fokusem	Obsługa menu klawiaturą i padem bez kodu	Automatyczny graf fokusu z geometrii, ui_accept / ui_cancel jako akcje	18
RichTextLabel z BBCode i efektami	Kolorowy, animowany tekst jednym stringiem	RichText z markupem [color=gold] , [wave] , [shake] , [img] , [link]	13, 18
CanvasLayer	HUD niezależny od kamery	Warstwy ekranowe UI i Overlay	12, 18
Tryby rozciągania (canvas_items , viewport , aspect keep/expand , skalowanie całkowite)	Jedna rozdzielczość bazowa działa na każdym ekranie	GameSettings.baseResolution , StretchMode , AspectMode , integerScaling	12
Sygnały	Reakcja na zdarzenie konkretnego obiektu	Eventy przypięte do obiektu: entity.on(...) , button.onClick(...)	8
Grupy (add_to_group , call_group)	Operacje na zbiorze obiektów	Tagi encji: tags().add("enemy") , world.query().tag("enemy")	14
Timer , await	Opóźnienia bez liczników	later(...) , Sequence	8
Tween (tween_property , równoległe, łańcuchy, callbacki)	Animacja dowolnej wartości jedną linijką	Tweens.to(entity, Props.POSITION, target, 0.5f).ease(Ease.OUT_BACK)	17
AnimationPlayer (ścieżki właściwości)	Choreografia wielu wartości w czasie	Timeline ze ścieżkami właściwości i zdarzeń	17
AnimatedSprite2D + SpriteFrames	Animacje klatkowe po nazwie	Animator z nazwanymi SpriteAnimation , import z Aseprite	17
CharacterBody2D.move_and_slide() , is_on_floor()	Ruch postaci z kolizjami bez własnej fizyki	Komponent Mover : moveAndSlide(velocity) , isOnFloor() , isOnWall() , platformy jednokierunkowe, snap do podłoża	15
Area2D z sygnałami wejścia i wyjścia	Strefy i wyzwalacze	Komponent Trigger + TriggerEnterEvent , TriggerExitEvent	15
Nazwane warstwy i maski kolizji	Kto z kim koliduje, bez bitów	Rejestr COLLISION_LAYER , .layer(PLAYER).collidesWith(WORLD, ENEMY)	15
RayCast2D , ShapeCast2D	Zapytania o linię wzroku i przeszkody	physics().raycast(...) , shapeCast(...) , overlap(...)	15

Godot	Co upraszcza	Gulp	Sekcja
Mapa wejść: <code>is_action_just_pressed</code> , <code>get_axis</code> , <code>get_vector</code>	Sterowanie niezależne od urządzenia	<code>input.justPressed(JUMP)</code> , <code>input.axis(LEFT, RIGHT)</code> , <code>input.vector(LEFT, RIGHT, UP, DOWN)</code>	16
Camera2D : wygładzanie, granice, marginesy, zoom	Kamera za graczem jedną liniijką	<code>camera.follow(player).smoothing(5).limits(ma.p.bounds()).deadZone(...)</code> , <code>shake(...)</code>	12
TileMapLayer : terrains (autotiling), kolizje i dane na kafelkach, animowane kafelki	Mapy bez ręcznego dobierania kafelków krawędzi	Terrain z automatycznym doбором, kształty kolizji i DataContainer w TileType	14
Parallax2D	Tło z efektem głębi	Parallax z warstwami i współczynnikiem przewijania	14
NavigationAgent2D	Szukanie drogi jedną metodą	Komponent NavAgent : <code>moveTo(target)</code>	15
Path2D + PathFollow2D	Ruch po krzywej	Path + komponent PathFollower	15
VisibleOnScreenNotifier2D	Reakcja na pojawienie się na ekranie	<code>EntityScreenEnterEvent</code> , <code>EntityScreenExitEvent</code> , <code>entity.isOnScreen()</code>	14
<code>process_mode</code> i pauza	Co działa podczas pauzy	<code>PauseMode.PAUSABLE / ALWAYS / WHEN_PAUSED</code> na komponentach, zadaniach i UI	8, 14
<code>change_scene_to_file</code>	Zmiana poziomu jedną liniijką	<code>worlds().switchTo("level2")</code> , <code>Transition.fade(0.4f)</code>	14
Autoload	Globalne systemy gry	GameModule i Services	8
PointLight2D, LightOccluder2D	Światło i cienie 2D	Light, Occluder, światło otoczenia świata	12
Widoczne kształty kolizji, zdalne drzewo sceny, regulacja szybkości gry	Debugowanie bez dodatkowego kodu	Nakładka F3, inspektor encji, <code>/timescale</code>	20
<code>move_toward</code> , <code>lerp</code> , <code>direction_to</code> , <code>angle_to</code>	Codzienna matematyka bez wzorów	Te same metody w Vec2 i Mathf	11
<code>tr()</code> z plikami tłumaczeń	Lokalizacja	<code>Text.translatable("menu.play")</code> , <code>tr("menu.play")</code>	19

Nie przejmujemy: edytora, plików scen, GDScriptu i drzewa węzłów dla logiki gry (logika opiera się na encjach i eventach). Drzewo węzłów występuje tylko w UI, gdzie jest naturalne.

11. Matematyka i narzędzia

Pakiet `dev.gulp.api.math` daje wszystko, czego potrzeba w 2D, z metodami nazwanymi po ludzku, tak by kod gry rzadko potrzebował wzorów. Każdy typ wartości ma wersję niemutowalną (rekord, do API) i mutowalną (do gorących pętli bez alokacji).

Typy

- `Vec2` (rekord) i `MutableVec2`: `add`, `sub`, `scale`, `dot`, `cross`, `length`, `lengthSquared`, `normalized`, `withX`, `withY`, `distanceTo`, `directionTo`, `angle`, `angleTo`, `rotated`, `lerp`, `moveToward(target, maxDelta)`, `clampLength`,

`perpendicular`, `reflect(normal)`, `snapped(step)`, `Vec2.fromAngle(deg)`, stałe `ZERO`, `ONE`, `UP`, `DOWN`, `LEFT`, `RIGHT`. Proponowany kierunek osi Y: w dół wszędzie, w świecie i w UI, jak w Godocie, Tiled i LDtk (decyzja w sekcji 23), więc `Vec2.UP = (0, -1)`.

- `Rect (x, y, width, height)`: `contains`, `overlaps`, `intersection`, `union`, `center`, `expand`, `fitInside`, `fitOutside` (do skalowania z zachowaniem proporcji).
- `Circle`, `Polygon` (z transformacją, `contains`, `area`, `centroid`), `Segment`, `Capsule`.
- `Transform2D` (pozycja, rotacja, skala, pochylenie) i `Mat3`; `Affine2` do batchera.
- `Color` (RGBA float): `rgb(0xFF8800)`, `hex("#ff8800")`, `hsv`, `lerp`, `withAlpha`, `mul`, predefiniowane kolory.
- `Grid` / `GridPos`: konwersje między współrzędnymi świata a komórkami, sąsiedzi (4 i 8 kierunków), linia Bresenhama, flood fill.

Funkcje

- `Mathf`: `clamp`, `lerp`, `inverseLerp`, `remap`, `smoothStep`, `moveToward`, `approach`, `wrap`, `pingPong`, `snapped`, `sign`, `isZero`, `nearlyEqual`, `lerpAngle`, `angleDifference`, `degToRad`, `damp(current, target, lambda, dt)` (wygładzanie niezależne od FPS).
- `Ease`: `LINEAR`, rodziny `SINE`, `QUAD`, `CUBIC`, `QUART`, `QUINT`, `EXPO`, `CIRC`, `BACK`, `ELASTIC`, `BOUNCE` w wariantach `IN`, `OUT`, `IN_OUT`, `OUT_IN`, plus `custom(f)` i `cubicBezier(x1, y1, x2, y2)`.
- Krzywe: `Bezier` (kwadratowa i sześcienna), `CatmullRom`, `BSpline`, z próbkowaniem po długości łuku.
- `Intersect`: punkt/odcinek/prostokąt/okrąg/wielokąt wzajemnie, SAT dla wielokątów wypukłych, odcinek-odcinek z punktem przecięcia.
- Geometria: triangulacja ear clipping, otoczka wypukła, upraszczanie linii, dekompozycja wielokąta wklęsłego na wypukłe.
- `Rng`: szybki generator z ziarnem (xoshiro256**), `nextInt(a, b)`, `nextFloat`, `chance(p)`, `pick(list)`, `weighted(map)`, `shuffle`, `insideCircle`, `onCircle`; niezależne instancje per świat dla determinizmu.
- `Noise`: Perlin i OpenSimplex2 w 1D, 2D i 3D, fraktalny (oktawy), domain warp; do generowania światów.

Narzędzia ogólne (w `gulp-core`, częściowo w API)

- `Pool<T>` z `obtain()` / `free()`, pule wbudowane dla `MutableVec2`, `Rect`, eventów częstych.
- Kolekcje prymitywne bez boxingu: `IntList`, `FloatList`, `IntIntMap`, `IntObjectMap`, `LongObjectMap` (dla chunków).

- `Logger` z poziomami i prefiksem modułu; na webie do konsoli przeglądarki.
- `Stopwatch`, `RollingAverage` do statystyk.

Wszystko w tym pakiecie ma testy właściwości (np. `normalized().length() == 1`) i nie alokuje w metodach typu mutowalnego.

12. Grafika: rendering 2D, kamera, skalowanie, światło

Grafika ma dwa poziomy: niski (`Texture`, `Shader`, `FrameBuffer`, `Mesh2D`) dla zaawansowanych i wysoki (`Draw`, komponenty sprite'ów, warstwy, kamera), który wystarcza do całej typowej gry. Wszystko rysuje jeden batcher, który sam łączy wywołania.

12.1 Potok klatki

1. Wyczyść ekran kolorem `clearColor`.
2. Dla każdego aktywnego świata i jego kamer: warstwy świata w kolejności z-order (parallax, warstwy mapy, encje, pierwszy plan), mapa światła, efekty post-process świata.
3. Warstwy ekranowe: UI (`Screen` i nakładki), `Overlay`, nakładka debug.
4. Skalowanie do okna według `Display` i prezentacja.

Eventy: `PreRenderEvent`, `RenderLayerEvent` (z `Draw` i nazwą warstwy, do rysowania własnego między warstwami), `PostRenderEvent`.

12.2 Niski poziom

- `Texture`: filtr (`NEAREST`, `LINEAR`, mipmapy), zawijanie (`CLAMP`, `REPEAT`, `MIRROR`), premultiplied alpha domyślnie, `upload(Pixmap)`, aktualizacja fragmentu.
- `TextureRegion`: fragment tekstury z UV, obrócenie z atlasu, przesunięcia po przycięciu przezroczystości.
- `Pixmap`: obraz w RAM z `getPixel/setPixel`, `fill`, `drawRect`, `drawCircle`, `drawLine`, `blit`, skalowaniem i zapisem PNG.
- `Shader`: źródło GLSL ES 3.00 z include'ami, typowane uniformy (`set("u_time", float)`), błędy kompilacji z numerem linii w logu.
- `Material`: shader + uniformy + tekstury dodatkowe + `BlendMode` (`NORMAL`, `ADD`, `MULTIPLY`, `SCREEN`, `PREMULTIPLIED`, `REPLACE`). Gotowe: `Materials.FLASH` (błysk trafienia), `OUTLINE`, `DISSOLVE`, `GRAYSCALE`, `TINT`.
- `FrameBuffer`: tekstura docelowa z opcjonalnym stencilem, `draw.into(fbo, d -> ...)`.

- `Mesh2D` : własne wierzchołki i indeksy, np. do terenów i efektów.

12.3 Batcher

- Jeden dynamiczny bufor wierzchołków (pozycja, UV, kolor spakowany, dodatkowy atrybut na flagi materiału), do 16 384 quadów na flush, podwójne buforowanie.
- Flush tylko przy zmianie tekstury, materiału, blendingu, clipu lub celu renderowania. Statystyki: `drawCalls`, `textureBinds`, `vertices` na klatkę.
- Sortowanie w warstwie po `zIndex`, a opcjonalnie po Y (`layer.ySort(true)` dla gier top-down).
- Cel wydajności: 10 000 animowanych sprite'ów przy 60 FPS w przeglądarce na średnim laptopie; 50 000 na desktopie.

12.4 API rysowania `Draw`

To samo API służy do rysowania własnego w świecie, w UI i w `Overlay`:

- Obrazy: `image(region, x, y)`, `image(region, x, y, w, h)`, `image(region, transform)`, `ninePatch(np, rect)`, `tiled(region, rect)`.
- Kształty (z antyaliasingiem): `rect`, `rectOutline`, `roundedRect(radius)`, `circle`, `circleOutline`, `ellipse`, `arc`, `line(a, b, width)`, `polyline`, `polygon`, `triangle`, `gradientRect`.
- Tekst: `text("Punkty: 10", x, y)`, `text(text, x, y, TextStyle)`, `text(text, rect, align)` z zawijaniem.
- Stan: `push()` / `pop()`, `translate`, `rotate`, `scale`, `color(Color)`, `alpha(float)`, `material(Material)`, `clip(rect, body)`, `into(fbo, body)`.

12.5 Warstwy renderowania

- Każdy świat ma listę `RenderLayer` z nazwą, `zOrder`, `ySort`, `parallaxFactor`, materiałem warstwy i widocznością. Domyślne: `background`, `tiles`, `entities`, `foreground`, `effects`.
- Encja ma `layer` i `zIndex`. Warstwy ekranowe (UI, overlay) nie podlegają kamerze.

12.6 Kamera

- `Camera`: `position`, `zoom`, `rotation`, `bounds()` (widoczny obszar), `screenToWorld`, `worldToScreen`.
- Podążanie: `follow(entity)`, `smoothing(float)` (wygładzanie niezależne od FPS), `deadZone(w, h)`, `lookAhead(float)`, `limits(Rect)`, `offset(Vec2)`.

- Efekty: `shake(trauma, seconds)` (model trauma: amplituda rośnie z kwadratem trauma), `zoomTo(z, seconds, Ease)`, `panTo(pos, seconds, Ease)`.
- Wiele kamer w świecie z obszarami ekranu (`viewportRect`) dla podzielonego ekranu i minimapy.

12.7 Skalowanie ekranu (`Display`)

Ustawienie	Wartości	Działanie
<code>baseResolution</code>	np. 640 × 360	Rozmiar, w którym projektuje się grę i UI
<code>StretchMode</code>	<code>DISABLED</code> , <code>CANVAS</code> , <code>VIEWPORT</code>	<code>CANVAS</code> : rysowanie w rozdzielczości okna ze skalowaniem współrzędnych (ostry tekst); <code>VIEWPORT</code> : rysowanie do bufora bazowego i skalowanie (pixel art)
<code>AspectMode</code>	<code>IGNORE</code> , <code>KEEP</code> , <code>KEEP_WIDTH</code> , <code>KEEP_HEIGHT</code> , <code>EXPAND</code>	Jak obsłużyć inne proporcje: pasy, rozciągnięcie lub pokazanie więcej świata
<code>integerScaling</code>	tak / nie	Skalowanie tylko o całkowite wielokrotności (pixel art bez rozmycia)
<code>pixelSnap</code>	tak / nie	Zaokrąglanie pozycji do pikseli bazowych
<code>letterboxColor</code>	kolor	Kolor pasów przy <code>KEEP</code>

HiDPI obsługiwane automatycznie: kod gry nigdy nie widzi pikseli fizycznych, chyba że zapyta `display().framebufferSize()`.

W trybie `VIEWPORT` świat rysuje się do bufora bazowego, ale UI, `Overlay` i tekst domyślnie w rozdzielczości okna, żeby tekst był ostry. `ui().pixelPerfect(true)` przenosi je do bufora bazowego dla spójnego pixel artu.

12.8 Post-processing

`world.postEffects()` i `display().postEffects()` przyjmują łańcuch efektów: `Bloom`, `Vignette`, `ColorGrade` (LUT), `Blur`, `Pixelate`, `ChromaticAberration`, `Crt`, `Custom(Shader)`. Każdy efekt ma właściwości animowalne tweenami.

12.9 Światło 2D

- `world.lighting().ambient(Color)` włącza system światła dla świata; wyłączony nie kosztuje nic.
- `Light: point, spot, directional`, z kolorem, promieniem, intensywnością, zanikiem i flagą cieni. Można je dopiąć do encji komponentem `LightSource`.
- Cienie z `Occluder` (kształty) i automatycznie z kafelków kolizyjnych; technika mapy cieni 1D per światło.
- Mapy normalnych dla sprite'ów jako opcja po 1.0.

13. Tekst, fonty i zasoby

Tekst rysuje się jedną metodą i jednym stylem, a zasoby ładuje się po kluczu `namespace:ścieżka` bez myślenia o systemie plików. Na desktopie w trybie deweloperskim zmiany plików są widoczne od razu (hot reload).

13.1 Fonty

Typ	Zastosowanie	Jak powstaje
<code>MsdffFont</code>	Domyślny dla UI i tekstu w świecie: ostry w każdym rozmiarze, obrys i cień w shaderze	Generowany z TTF/OTF przez <code>packAssets</code> (<code>msdfgen</code>)
<code>BitmapFont</code>	Pixel art, fonty ręcznie rysowane	Format BMFont (<code>.fnt</code> + PNG) lub siatka znaków z obrazka
<code>DynamicFont</code>	Znaki nieznane w czasie buildu (czat, imiona, CJK)	Rasteryzacja w locie: FreeType na desktopie, Canvas2D na webie (plik fontu ładowany przez <code>FontFace API</code>), cache stron glifów

- `FontFamily` łączy warianty regular, bold, italic, bold-italic. Łańcuch fallbacków dla brakujących znaków (np. `MsdffFont` → `DynamicFont`).
- Zestaw znaków domyślny: łaciński rozszerzony (w tym polskie znaki), cyrylica, greka. Font domyślny wbudowany w silnik na licencji OFL.
- W 1.0: pisma od lewej do prawej z kerningiem. Pełne kształtowanie (RTL, ligatury pism złożonych) po 1.0 — decyzja w sekcji 23.

13.2 Styl, układ i tekst sformatowany

- `TextStyle` (budowniczy): `font`, `size`, `color`, `outline(width, color)`, `shadow(dx, dy, color)`, `letterSpacing`, `lineHeight`, `bold`, `italic`.
- `TextLayout`: pomiar i układ z zawijaniem (po słowach lub znakach), wyrównaniem, limitem linii i wielokropkiem. Cache'owany, dopóki tekst i styl się nie zmienia.
- `Text` to model tekstu sformatowanego (w stylu komponentów Adventure):
`Text.of("Monety: ").append(Text.of("10").color(Color.GOLD).bold()),`
`Text.translatable("hud.coins", 10).`
- Markup w stringach: `[b]`, `[i]`, `[color=gold]`, `[size=20]`, `[font=coins:pixel]`, `[img=coins:icons/coin]`, `[link=shop]` (kliknięcie wywołuje `TextLinkClickEvent`), efekty animowane `[wave]`, `[shake]`, `[rainbow]`, `[pulse]`, `[fade]`. Parser własny, błędy markupu wyświetlane jako tekst z ostrzeżeniem w logu.
- Efekt maszyny do pisania: właściwość `visibleCharacters` animowana tweenem, event `TextRevealCompleteEvent`.

13.3 Zasoby: struktura i klucze

```
assets/<namespace>/
  textures/ sprites/ fonts/ sounds/ music/ shaders/
  maps/ animations/ particles/ lang/ data/ config/
```

- Klucz zasobu to `AssetKey<T> = Key + typ`, np.
`AssetKey.texture("coins:sprites/player")`. Rozszerzenie pliku jest opcjonalne.
- `generateAssetKeys` tworzy klasę `GameAssets` ze stałymi
`(GameAssets.Sprites.PLAYER)`, więc literówki w ścieżkach są błędami kompilacji.

13.4 Assets

- `get(key)` zwraca załadowany zasób lub rzuca czytelny wyjątek. `load(key)` zwraca `Promise<T>`. `isLoading`, `unload`, `progress()`.
- Grupy: `assets().group("level1").add(...).addFolder("coins:sprites/level1")`, potem `loadGroup("level1")` z postępem. Grupa `startup` ładuje się przed `onStart()` z wbudowanym ekranem ładowania, który gra może podmienić.
- Liczenie referencji i zależności (atlas → tekstura, font → tekstura, mapa → zestawy kafelków). Zasób znika, gdy nic go nie używa.
- Własne typy: `AssetLoader<T>` rejestrowany w `onLoad`.
- Eventy: `AssetLoadEvent`, `AssetGroupLoadedEvent`, `AssetReloadEvent`, `AssetLoadFailedEvent`.

13.5 Atlasy

- `packAssets` pakuje każdy folder `sprites/` w atlasy (strony do 4096×4096 , padding, extrude krawędzi przeciw krwawieniu, przycinanie przezroczystości, wiele stron). Regiony dostępne po oryginalnej ścieżce pliku:
`assets().region("coins:sprites/player/idle_0")`.
- W trybie deweloperskim pakowanie odbywa się w locie przy starcie, więc nie trzeba uruchamiać kroku buildu.
- Zgodność z formatem JSON TexturePackera i eksportem Aseprite (arkusz + tagi animacji).

13.6 Formaty

Obrazy: PNG, JPG, WebP. Audio: OGG Vorbis, WAV. Fonty: TTF, OTF, BMFont, MSDF (JSON + PNG). Mapy: Tiled `.tmj`, LDtk `.ldtk`. Animacje: Aseprite JSON. Dane: JSON, YAML (konfiguracja).

13.7 Hot reload i resource packi

- Desktop, tryb deweloperski: obserwacja `assets/` i `config/`. Tekstury, atlasy, shadery, fonty, tłumaczenia, dane i konfiguracja podmieniają się w miejscu (te same uchwytty), a potem wywoływany jest `AssetReloadEvent`. Mapy tylko wywołują event, a gra decyduje, czy przeładować świat. Kod: standardowy HotSwap JVM z IDE (zmiany ciał metod bez restartu); zmiany sygnatur wymagają restartu.
- Web, tryb deweloperski: serwer `runWeb` odświeża stronę po zmianie kodu lub zasobów.
- Resource packi: foldery lub ZIP w `resourcepacks/`, nadpisują zasoby po kluczu, kolejność ustala gracz lub gra, zmiana w locie wywołuje `ResourcePackChangeEvent`. Na webie packi są dołączane w buildzie.

14. Świat, encje, komponenty i mapy kafelkowe

Logika gry opiera się na światach, encjach i komponentach, tak jak serwer Minecrafta opiera się na światach, encjach i blokach. Encja to obiekt w świecie, typ encji definiuje jej skład, a komponenty dodają zachowania i dane.

14.1 Worlds i World

- `worlds().create(name, WorldSettings)`, `load(name, WorldSource)` (Tiled, LDtk, generator, zapis), `register(name, WorldSource)` (leniwie: ładuje się przy pierwszym `switchTo`), `get(name)`, `active()`, `switchTo(name, Transition)`, `unload(name)`. Wiele światów naraz jest dozwolone; nieaktywne mogą być wstrzymane lub tickowane.

- `WorldSettings`: `gravity(Vec2)`, `tileSize`, `bounds`, `ambientLight`, `seed`, `persistent`.
- `World`: `spawn(type, x, y)`, `spawn(type, location, e -> {...})` (konfiguracja przed dodaniem, jak w Bukkie), `entities()`, `entity("boss")` (po nazwie), `query()`, `tileMap()`, `camera()`, `cameras()`, `layers()`, `physics()`, `lighting()`, `postEffects()`, `parallax()`, `rng()`, `data()`, `ticks()`, `playSound(pos, sound)`, `spawnParticles(effect, pos)` (pozycja jako `Vec2` albo `Location`), `raycast(...)`, `entitiesNear(pos, radius)`, `entitiesIn(rect)`.
- `Location` (rekord: świat, x, y) z metodami jak `Vec2`.
- Przejścia (`Transition`, rejestr): `fade`, `fadeColor`, `slide`, `circleWipe`, `pixelate`, `shader(Shader)`.
- Eventy: `WorldLoadEvent`, `WorldUnloadEvent`, `WorldSwitchEvent`, `WorldSaveEvent`.

14.2 Entity

- Tożsamość: `id()` (UUID), `runtimeId()` (int), `type()`, `world()`, `name()` (opcjonalna, unikalna w świecie).
- Transformacja: `position()`, `setPosition`, `teleport(location)`, `rotation`, `scale`, `size`, `bounds()`. Pozycja renderowana jest interpolowana automatycznie; `teleport` i pierwsza klatka po spawnie pomijają interpolację, żeby encja nie przelatwała przez ekran.
- Wygląd: `layer`, `zIndex`, `visible`, `tint`, `flipX`, `flipY`.
- Hierarchia: `attach(child, offset)` i `detach()` — dziecko podąża za rodzicem (jak pasażerowie w Minecrafcie i dzieci węzłów w Godocie). Usunięcie rodzica usuwa dzieci, chyba że `detachOnRemove`.
- Komponenty: `get(Class)`, `find(Class)` (Optional), `has`, `add(component)`, `remove(Class)`.
- `tags()`, `data()` (`DataContainer`), `persistent` (zapisywana ze światem), `pauseMode`.
- `on(EventClass, handler)` — eventy tej encji.
- `remove()` (usunięcie na końcu ticku), `isRemoved()`, `isOnScreen()`.

14.3 EntityType

```

ENEMY = registries().register(Registries.ENTITY_TYPE,
EntityType.builder(key("slime"))
    .size(0.8f, 0.6f)
    .component(() -> new SpriteComponent(GameAssets.Sprites.SLIME))
    .component(() -> new Animator(GameAssets.Animations.SLIME))
    .component(() -> new Mover().gravity(true))
    .component(() -> new Health(10))
    .component(SlimeAi::new)
    .tags("enemy")
    .layer("entities")
    .build());

```

Komponenty podaje się jako fabryki, więc każda encja ma własne instancje.

`.parent(BASE_ENEMY)` dziedziczy konfigurację innego typu.

14.4 Component

```

public abstract class Component {
    protected void onAttach() {} // dodany do encji (jeszcze nie w świecie)
    protected void onSpawn() {} // encja w świecie
    protected void onTick() {} // co tick, jeśli enabled
    protected void onRemove() {} // encja usuwana
    public final Entity entity() { ... }
    public final World world() { ... }
    public final Scheduler scheduler() { ... } // zadania anulowane z encją
    public int tickOrder() { return 0; } // kolejność w ticku
}

```

- Komponenty trwałe: `@ComponentInfo(key = "coins:wallet", persistent = true)` i pola z `@Save` — procesor generuje codec do zapisu.
- Komponenty wbudowane: `SpriteComponent`, `Animator`, `Collider`, `Body`, `Mover`, `Trigger`, `Health`, `LightSource`, `SoundEmitter`, `ParticleEmitter`, `NavAgent`, `PathFollower`, `Lifetime` (usuwa encję po czasie), `Follow`, `WorldText` (tekst nad encją), `WorldUi` (węzeł UI przypięty do encji, np. pasek życia), `Interactable` (klik i hover w świecie), `StateMachine`, `BehaviorTree`.
- `Health`: `max`, `current`, `damage(amount, DamageType, source)`, `heal`, `invulnerableTicks`, `isDead`. Wywołuje `EntityDamageEvent` (anulowalny, z modyfikowalną wartością), `EntityHealEvent`, `EntityDeathEvent`.

14.5 Zapytania

`world.query().with(Health.class).tag("enemy").near(player.position(), 8).filter(e -> ...).forEach(...)`, a także `first()`, `count()`, `list()`, `sortedByDistance(pos)`. Zapytania korzystają z indeksów komponentów, tagów i siatki przestrzennej.

Eventy encji: `EntitySpawnEvent` (anulowalny), `EntityRemoveEvent`, `EntityTeleportEvent`, `EntityClickEvent`, `EntityHoverEnterEvent`, `EntityHoverExitEvent`, `EntityScreenEnterEvent`, `EntityScreenExitEvent`, eventy zdrowia jak wyżej, eventy kolizji w sekcji 15.

Cele wydajności: 20 000 aktywnych encji z prostymi komponentami przy 60 TPS na desktopie, 5 000 w przeglądarce. Wewnętrznie komponenty tego samego typu są trzymane w tablicach dla szybkiej iteracji.

14.6 Mapy kafelkowe

- `TileMap` w świecie: orientacje `ORTHOGONAL`, `ISOMETRIC`, `ISOMETRIC_STAGGERED`, `HEXAGONAL` (pointy i flat); `tileSize`; warstwy `TileLayer` (nazwa, z-order, widoczność, kolizyjna, parallax, tint).
- Operacje: `tile(layer, x, y)`, `setTile(layer, x, y, type)` (wywołuje anulowalny `TileChangeEvent`), `fill(layer, rect, type)`, `clear`, `worldToTile`, `tileToWorld`, `bounds()`.
- `TileType` (rejestr): region tekstury lub kafelek z `TileSet`, kształt kolizji (`FULL`, `NONE`, `SLOPE_LEFT`, `SLOPE_RIGHT`, `HALF_TOP`, `ONE_WAY`, własny wielokąt), `friction`, animacja klatkowa, `data()`, `onInteract`, oraz tick: `tickRandomly(chance)` (jak losowe ticki bloków w Minecrafcie, np. wzrost trawy) i `tickEvery(n)`.
- Kafelki ze stanem (`TileState`, np. skrzynia z zawartością): przechowywane rzadko, z `DataContainer`, zapisywane ze światem.
- `Terrain` (autotiling): zestawy terenów w trybach 16 i 47 kafelków oraz dopasowanie po rogach i bokach. `setTerrain(layer, x, y, terrain)` i `paintTerrain(rect, terrain)` same dobierają krawędzie i narożniki sąsiadów.
- Chunki 32×32 : ładowane wokół kamer, gracza i obszarów `world.keepLoaded(rect, owner)`; eventy `ChunkLoadEvent`, `ChunkUnloadEvent`; siatki renderowania budowane raz na chunk i przebudowywane przy zmianie.
- `ChunkGenerator`: `generate(ChunkData data, int cx, int cy, Rng rng)`, uruchamiany poza tickiem (wątki na desktopie, kolejka między klatkami na webie), z możliwością spawnu encji po wygenerowaniu.

- Importy: Tiled (warstwy kafelków, warstwy obiektów, właściwości → `DataContainer`, animowane kafelki) i LDtk (poziomy, `IntGrid`, auto-warstwy, encje z polami, światy wielopoziomowe). Obiekty map zamieniają się w encje przez `ObjectSpawner`:
`.map("slime", ENEMY).`

14.7 Parallax

`world.parallax().layer(GameAssets.Textures.SKY, 0.1f).repeatX().autoScroll(-4, 0)` — warstwy tła z współczynnikiem przewijania, powtarzaniem i automatycznym ruchem.

15. Fizyka, kolizje, nawigacja i AI

Fizyka ma dwa poziomy: kinematyczny w stylu Godota (ruch postaci jedną metodą, wystarczający dla większości gier 2D) oraz pełną fizykę brył sztywnych, która zastępuje `Box2D` z `libGDX`. Oba są w czystej Javie, bo muszą działać pod `TeaVM`.

15.1 Warstwy kolizji

Warstwy to klucze w rejestrze `COLLISION_LAYER` (maksymalnie 32). Collider ma `layer(PLAYER)` i `collidesWith(WORLD, ENEMY, PICKUP)`. Macierz kolizji można też ustawić globalnie: `physics().setCollides(PLAYER, ENEMY, true)`.

Domyślnie collider jest na warstwie `gulp:default` i koliduje ze wszystkimi warstwami. `Trigger` reaguje na encje, których warstwa jest w jego masce.

15.2 Poziom kinematyczny

- `Collider`: kształt (`box`, `circle`, `capsule`, `polygon`), przesunięcie, warstwa, maska. Kafelki kolizyjne mapy są kolizjami statycznymi automatycznie. Jeśli encja z `Mover`, `Trigger` lub `Body` nie ma `Collider`, używany jest prostokąt o rozmiarze encji (`size`).
- `Mover` (odpowiednik `CharacterBody2D`):
 - `moveAndSlide(Vec2 velocity)` przesuwa encję, ślizga się po ścianach i zboczach, zwraca listę kontaktów.
 - `isOnFloor()`, `isOnWall()`, `isOnCeiling()`, `floorNormal()`, `wallNormal()`.
 - Ustawienia: `gravity(boolean)`, `maxFloorAngle(deg)`, `floorSnap(distance)`, `stepHeight`, `platformCarry` (unoszenie przez ruchome platformy), `oneWayPlatforms`.
 - Pomocnicze dla platformówek: `coyoteTicks(n)`, `jumpBufferTicks(n)`, `canJump()`, `jump(speed)`, `dropThroughPlatform()`.
 - Tryb top-down: `topDown(true)` wyłącza grawitację i pojęcie podłogi.

- `Trigger` : strefa bez fizycznej reakcji, eventy `TriggerEnterEvent` , `TriggerExitEvent` , `triggered()` (lista encji w środku).
- Algorytm: faza szeroka na siatce przestrzennej, faza wąska SAT, rozwiązywanie osi X i Y osobno, podkroki przy dużej prędkości, by nie przelatywać przez ściany.

15.3 Fizyka brył sztywnych

- `Body` z `BodyType.DYNAMIC` , `KINEMATIC` , `STATIC` ; kształty: okrąg, wielokąt wypukły (do 8 wierzchołków), kapsuła, odcinek, łańcuch; wielokąty wklęsłe dzielone automatycznie.
- Właściwości: masa lub gęstość, tarcie, sprężystość, tłumienie liniowe i kątowe, `fixedRotation` , `bullet` (ciągłe wykrywanie kolizji), `gravityScale` , usypianie ciał.
- Siły: `applyForce` , `applyImpulse` , `applyTorque` , `setVelocity` , `setAngularVelocity` .
- Złącza: `revolute` , `prismatic` , `distance` , `weld` , `wheel` , `mouse` , `motor` , `rope` , z limitami i silnikami.
- Solver pisany od zera na podstawie publicznie opisanych algorytmów (impulsy sekwencyjne z podkrokami). Stały krok równy tickowi gry, domyślnie 4 podkroki.
- Kontakty: `EntityCollideEvent` (początek, z punktem, normalną i siłą impulsu), `EntityCollideEndEvent` , `PreCollideEvent` (anulowalny, np. platforma jednokierunkowa).

15.4 Zapytania

`physics().raycast(from, to, mask)` → `RayHit` (encja, punkt, normalna, ułamek) lub pusty; `raycastAll`; `shapeCast(shape, from, to, mask)`; `overlapPoint` , `overlapCircle` , `overlapRect` . Dostępne w obu poziomach fizyki. Eventy wspólne: `EntityLandEvent` (dotknięcie podłogi po locie), `EntityCollideEvent` .

15.5 Nawigacja

- `NavGrid` budowany automatycznie z kafelków (przechodność i koszt w `TileType`), z dynamicznymi przeszkodami (`navGrid.block(rect, owner)`).
- `PathFinder` : A* z ruchem 4 lub 8 kierunków, wygładzanie ścieżki, Jump Point Search dla dużych siatek, pola przepływu dla wielu jednostek z jednym celem, oraz A* na dowolnym grafie (`Graph<N>`).
- `NavAgent` : `moveTo(target)` , `speed` , `arrivalDistance` , `repathInterval` , proste omijanie innych agentów; eventy `NavTargetReachedEvent` , `NavPathFailedEvent` . Na webie obliczenia ścieżek są rozłożone na klatki.
- `Path` (z punktów, krzywej lub polilinii z `Tiled/LDtk`) i `PathFollower` : prędkość, zapętlenie, ping-pong, obrót wzdłuż ścieżki, event `PathEndEvent` .

15.6 AI

- `StateMachine<S>`: stany z `enter`, `tick`, `exit`, przejścia warunkowe, `changeState`, event `StateChangeEvent`.
- `BehaviorTree`: budowniczy z węzłami `sequence`, `selector`, `parallel`, dekoratorami `inverter`, `repeat`, `cooldown`, `timeout`, `untilSuccess`, warunkami i akcjami zwracającymi `RUNNING`, `SUCCESS`, `FAILURE`. Tablica (blackboard) to `DataContainer` encji.
- `Steering`: `seek`, `flee`, `arrive`, `wander`, `pursue`, `evade`, `separation`, `cohesion`, `alignment`, `followPath`, `avoidObstacles`, łączone wagami w komponencie `Steering`.
- Komunikacja między agentami odbywa się przez eventy (odpowiednik dyspozytora wiadomości z `gdx-ai`).

16. Input i audio

Sterowanie opiera się na akcjach niezależnych od urządzenia (jak mapa wejść w Godocie), a dźwięk na kluczach z rejestru i szynach głośności. Kod gry prawie nigdy nie pyta o konkretny klawisz.

16.1 Akcje wejścia

```
JUMP = registries().register(Registries.INPUT_ACTION,
    InputAction.builder(key("jump"))
        .bind(Keys.SPACE, Keys.W, GamepadButton.SOUTH)
        .build());
MOVE_LEFT = InputAction.builder(key("move_left"))
    .bind(Keys.A, Keys.LEFT, GamepadAxis.LEFT_X.negative(),
    GamepadButton.DPAD_LEFT)
    .build();
```

- Odczyt w ticku: `input.pressed(JUMP)`, `justPressed`, `justReleased`, `heldTicks(JUMP)`, `strength(action)` (0-1 dla analogów), `axis(MOVE_LEFT, MOVE_RIGHT)` (-1...1), `vector(LEFT, RIGHT, UP, DOWN)` (znormalizowany, z okrągłą martwą strefą).
- Stany są liczone per tick, nie per klatkę, więc krótkie naciśnięcie między tickami nie ginie.
- Zestawy akcji: `ActionSet` (np. „gameplay”, „menu”) włączane i wyłączane razem; otwarcie `Screen` z `blocksGameplayInput` wyłącza zestaw `gameplay` automatycznie.

- Zmiana przypisań: `input.bindings().rebind(action, slot, binding)`, `captureNextInput(callback)` dla menu ustawień, wykrywanie konfliktów, automatyczny zapis w `Preferences`.
- Nazwy i ikony przypisań: `binding.displayName()` i `binding.glyph()` dobierają ikonę do wykrytego kontrolera (Xbox, PlayStation, Nintendo, klawiatura).
- Eventy akcji: `ActionPressEvent`, `ActionReleaseEvent`.

16.2 Urządzenia

- Klawiatura: `isDown(Keys.A)`, eventy `KeyPressEvent`, `KeyReleaseEvent`, `KeyRepeatEvent`, `CharTypedEvent`; kody fizyczne (scancode) dla układów innych niż QWERTY.
- Mysz: pozycja na ekranie i w świecie (`input.mouseWorld(camera)`), przesunięcie, przyciski, scroll; eventy `MouseButtonPressEvent`, `MouseButtonReleaseEvent`, `MouseMoveEvent` (najwyżej raz na klatkę), `MouseScrollEvent`. Kursory: systemowe (strzałka, ręka, tekst, zmiana rozmiaru), własne z regionu tekstury, tryby `NORMAL`, `HIDDEN`, `CAPTURED`.
- Dotyk (web na urządzeniach dotykowych): `input.touches()`, mapowanie pierwszego dotyku na mysz; gesty `tap`, `doubleTap`, `longPress`, `pan`, `pinch`, `swipe` jako eventy; wirtualny joystick w UI.
- Gamepady: podłączanie w trakcie gry (`GamepadConnectEvent`, `GamepadDisconnectEvent`), wielu graczy `input.gamepad(i)`, standardowe nazwy przycisków (`SOUTH`, `EAST`, ...), martwe strefy, wibracje tam, gdzie platforma pozwala.
- Kolejność obsługi: konsola deweloperska → UI (fokus i hover) → listenery gry. Event skonsumowany przez UI ma `isConsumedByUi() == true`, a akcje gameplay go nie widzą.
- Tekst: pole tekstowe UI obsługuje IME; na webie używa ukrytego elementu `input`, by pokazać klawiaturę ekranową. Schowek: `input.clipboard().get()`, `set(text)`.

16.3 Audio

- `Sound` (krótki, zdekodowany w pamięci) rejestrowany w `SOUND` z wariantami (losowy plik), losowym zakresem głośności i wysokości, limitem jednoczesnych instancji i minimalnym odstępem (żeby 30 monet naraz nie dało 30 dźwięków).
- Odtwarzanie: `audio().play(sound)`, `play(sound, volume, pitch)`, `world.playSound(location, sound)` z tłumieniem odległości od słuchacza (domyślnie aktywna kamera) i panoramą stereo. Zwraca `Playback: stop`, `fadeOut(seconds)`, `volume`, `pitch`, `loop`, `isPlaying`.

- Szyny: `master`, `music`, `sfx`, `ui`, `voice` + własne; głośność, wyciszenie, filtry dolno- i górnoprzepustowe (np. stłumienie przy pauzie), pogłos, automatyczne ściszenie muzyki przy głoście.
- Muzyka strumieniowana: `audio().music().play(music, fadeInSeconds)`, `crossfadeTo(other, seconds)`, punkty pętli (początek i koniec pętli w próbkach), playlisty; event `MusicEndEvent`.
- Limit głosów (domyślnie 32) z priorytetami: przy braku miejsca cichnie najcichszy i najstarszy dźwięk.
- `PauseMode` dla dźwięków: dźwięki gry pauzują się z grą, dźwięki UI nie.
- Surowe PCM: `audio().stream(PcmSource)` do dźwięku proceduralnego (odpowiednik `AudioDevice`). Nagrywanie z mikrofonu tylko na desktopie, jako opcja.
- Web: odblokowanie audio po pierwszym geście obsługiwane automatycznie; dźwięki zagrane przed odblokowaniem są pomijane, muzyka startuje po odblokowaniu.

17. Animacje, tweeny i cząsteczki

Każdą wartość da się animować jedną linijką (tween), animacje klatkowe odtwarza się po nazwie, a cząsteczki spawnuje jak w Bukkie. Wszystkie animacje należą do właściciela (encja, węzeł UI, moduł) i znikają razem z nim.

17.1 Właściwości animowalne

`Property<T, V>` to typowany dostęp do wartości (getter, setter, interpolator), bez refleksji. Wbudowane w klasie `Props`:

- Encja: `POSITION`, `X`, `Y`, `ROTATION`, `SCALE`, `ALPHA`, `TINT`.
- Kamera: `CAMERA_POSITION`, `CAMERA_ZOOM`, `CAMERA_ROTATION`.
- UI: `NODE_OFFSET`, `NODE_SIZE`, `NODE_ALPHA`, `NODE_SCALE`, `NODE_ROTATION`, `NODE_COLOR`.
- Inne: `VOLUME` (odtworzenie i szyny), `LIGHT_INTENSITY`, `LIGHT_RADIUS`, `LIGHT_COLOR`, parametry efektów post-process.
- Własne: `Property.of(Enemy::speed, Enemy::setSpeed, Interpolators.FLOAT)`.

17.2 Tweeny

```
Tweens.to(coin, Props.POSITION, hudCoinPos, 0.4f).ease(Ease.IN_BACK)
    .then(Tweens.to(coin, Props.SCALE, 0f, 0.1f))
    .onComplete(coin::remove)
    .start();
```

- Tworzenie: `to` (do wartości), `from`, `by` (względnie); ustawienia `ease`, `delay`, `repeat(n)`, `yoyo()`, `onUpdate`, `onComplete`, `realtime()` (ignoruje pauzę i `timeScale`).
- Kompozycja: `then(...)`, `Tweens.sequence(...)`, `Tweens.parallel(...)`, `Tweens.wait(seconds)`, `Tweens.call(runnable)`.
- Sterowanie: `pause`, `resume`, `kill`, `Tweens.killAll(target)`, `isRunning`, `progress`.
- Gotowe efekty: `Tweens.punchScale(target, 0.2f, 0.3f)`, `flash(entity, Color.WHITE, 0.1f)`, `fadeIn`, `fadeOut`, `pulse`, `bob` (unoszenie się), `shake(target, strength, seconds)`.

17.3 Timeline

Choreografia wielu właściwości w czasie (odpowiednik `AnimationPlayer`): ścieżki klatek kluczowych dla właściwości, zdarzenia w punktach czasu (`at(0.5f, () -> ...)`), długość, pętla, prędkość, `play`, `stop`, `seek`, `reverse`. Definiowana w kodzie budowniczym.

17.4 Animacje klatkowe

- `SpriteAnimation`: klatki (regiony), czas każdej klatki, tryb (`ONCE`, `LOOP`, `PING_PONG`, `REVERSED`, `LOOP_RANDOM`), zdarzenia na klatkach (`onFrame(3, ...)`, np. dźwięk kroku).
- `AnimationSet` (zasób): zestaw nazwanych animacji, importowany z Aseprite (tagi → nazwy animacji, czasy klatek, kierunek tagu) lub budowany z regionu atlasu po wzorcu nazw (`player/run_0..7`).
- Komponent `Animator`: `play("run")`, `playOnce("attack").then("idle")`, `speed`, `current()`, `isFinished()`; eventy `AnimationEndEvent`, `AnimationFrameEvent`.
- Automatyczny wybór animacji z warunków (lekki odpowiednik `AnimationTree`): `animator.auto().when(() -> !mover.isOnFloor(), "jump").when(() -> mover.velocity().x() != 0, "run").otherwise("idle")`.
- Animacje szkieletowe (Spine, DragonBones) poza 1.0.

17.5 Cząsteczki

- `ParticleEffect` budowany w kodzie lub z JSON w `particles/` (hot reload), złożony z emiterów.
- Emiter: emisja (`rate` na sekundę, `burst(count, atSeconds)`, czas trwania, pętla), kształt (`point`, `circle`, `ring`, `rect`, `line`), kierunek i rozrzut kątowy, zakres prędkości, grawitacja, opór, prędkość kątowa, czas życia, krzywe rozmiaru, koloru

(gradient) i przezroczystości w czasie życia, region lub animacja, `BlendMode`, przestrzeń lokalna lub świata, pod-emiterzy przy śmierci cząstki, opcjonalna kolizja z kafelkami (odbicie lub zniknięcie).

- Spawn: `world.spawnParticles(effect, location)` zwraca `ParticleInstance` (`stop`, `follow(entity)`); komponent `ParticleEmitter` dla efektów przypiętych do encji.
- Symulacja na CPU z pulami, limit na świat (domyślnie 20 000 cząstek), rysowanie przez batcher.
- Przykład `showcase` zawiera plac zabaw cząsteczek z hot reloadem JSON zamiast osobnego edytora.

18. UI bez matematyki

Interfejs buduje się w kodzie z kontenerów i widgetów, a pozycje i rozmiary liczy silnik — kod gry nie zawiera żadnej współrzędnej poza opcjonalnymi odstępami. Są trzy poziomy: `Screen` (pełne ekrany jak menu), HUD (stałe elementy na ekranie gry) i `Overlay` (rysowanie natychmiastowe, np. tekst w dowolnym miejscu).

18.1 Przykład

```
import static dev.gulp.api.ui.Ui.*;

public final class MainMenu extends Screen {
    @Override protected Node build() {
        return center(
            column(
                label(tr("menu.title")).variant("title"),
                button(tr("menu.play")).onClick(() ->
worlds().switchTo("level1", Transition.fade(0.4f))),
                button(tr("menu.settings")).onClick(() -> ui().push(new
SettingsScreen()))),
                button(tr("menu.quit")).onClick(() -> engine().stop())
            ).gap(12).width(240)
        );
    }
}
```

Przyciski są jeden pod drugim, wyśrodkowane w oknie o dowolnej rozdzielczości, obsługiwane myszą, klawiaturą i padem, bez jednej liczby określającej pozycję.

18.2 Node — wspólne właściwości

- Rozmiar: `width`, `height`, `minSize`, `maxSize`; bez podania rozmiar wynika z zawartości.
- Flagi w kontenerze (osobno dla osi X i Y): `.fill()`, `.expand()`, `.expand(ratio)`, `.shrinkStart()`, `.shrinkCenter()`, `.shrinkEnd()`. Skróty: `.grow()` = `expand` + `fill` w osi kontenera.
- Pozycjonowanie poza kontenerem (w `Stack`, HUD, korzeniu ekranu):
`.anchor(Anchor.TOP_RIGHT)`, `.offset(8, 8)`, `.fillParent()`,
`.fillParent(Insets.all(16))`. Presety `Anchor`: 9 punktów (rogi, krawędzie, środek) oraz `TOP_WIDE`, `BOTTOM_WIDE`, `LEFT_WIDE`, `RIGHT_WIDE`, `FULL`.
- Wygląd: `visible`, `alpha`, `scale`, `rotation` (tylko wizualne, z punktem obrotu, nie wpływa na layout), `variant("name")`, `style(s -> s.background(...))`.
- Interakcja: `enabled`, `focusable`, `mouseFilter` (`STOP`, `PASS`, `IGNORE`), `tooltip(...)`, `cursor(...)`.
- Identyfikacja: `id("play")`, `find("play")`, `findAll(Button.class)`.
- Zdarzenia: `onClick`, `onHover`, `onFocus`, `onChange`, `onSubmit`, ogólnie `on(EventClass, handler)`.

18.3 Kontenery

Kontener	Układ
<code>column(...)</code> , <code>row(...)</code>	Dzieci w pionie / poziomie; <code>gap</code> , <code>align</code> w osi poprzecznej; wolne miejsce dzielone między dzieci z <code>expand</code> według współczynników
<code>grid(columns, ...)</code>	Siatka o stałej liczbie kolumn; <code>hGap</code> , <code>vGap</code> ; flagi <code>expand</code> działają w obu osiach
<code>stack(...)</code>	Dzieci na sobie; każde pozycjonowane kotwicą
<code>center(child)</code>	Wyśrodkowanie dziecka w minimalnym rozmiarze
<code>margin(insets, child)</code>	Marginesy wokół dziecka
<code>panel(child)</code>	Tło z motywu + wewnętrzny odstęp
<code>scroll(child)</code>	Przewijanie w pionie i/lub poziomie, paski wg polityki, podjazd do elementu z fokusem, przewijanie kółkiem, dotykiem i padem

Kontener	Układ
<code>split(a, b)</code>	Dwa panele z przeciąganą granicą
<code>flow(...)</code>	Układ z zawijaniem do następnej linii
<code>tabs(tab("Grafika", ...), tab("Dźwięk", ...))</code>	Zakładki
<code>foldable("Tytuł", child) + FoldGroup</code>	Rozwijana sekcja, grupa pozwala na jedną otwartą
<code>aspect(ratio, child)</code>	Zachowanie proporcji dziecka
<code>spacer()</code>	Pusty element z <code>expand</code> , rozpycha sąsiadów

18.4 Widgety

`label`, `richText` (markup z sekcji 13), `image`, `button` (tekst, ikona lub oba), `iconButton`, `checkbox`, `radio` z `ButtonGroup`, `toggle` (przełącznik), `slider`, `progressBar` (poziomy i kołowy), `spinBox`, `textField` (walidacja, maska, placeholder, hasło), `textArea`, `dropdown`, `listView` (wirtualizowany, zaznaczanie pojedyncze i wielokrotne), `tree`, `dialog` i `modal`, `window` (przeciągane, zamykane), `colorPicker`, `itemGrid` (sloty z przeciąganiem, np. ekwipunek), `separator`, `keybindButton` (przechwytuje nowe przypisanie akcji), `virtualJoystick`, `toast` (krótkie powiadomienia w rogu ekranu).

18.5 Algorytm layoutu (do implementacji)

- Pomiar (od dołu):** każdy węzeł liczy minimalny rozmiar z zawartości (tekst, obraz, dzieci + odstępy + padding stylu), ograniczony `minSize` i `maxSize`.
- Rozmieszczenie (od góry):** kontener dostaje prostokąt i dzieli go. W `row / column`: wolne miejsce = rozmiar – suma minimów – odstępy; dzieli się je między dzieci z `expand` proporcjonalnie do współczynników, z ponownym podziałem, gdy ktoś osiągnie `maxSize`. W osi poprzecznej `fill` rozciąga, a `shrink*` wyrównuje.
- Kotwice:** węzeł poza kontenerem ma cztery kotwice (0–1) względem rodzica i przesunięcia; preset `Anchor` ustawia je jednym wywołaniem.
- Unieważnianie:** zmiana tekstu, rozmiaru, widoczności lub dzieci oznacza węzeł i przodków jako brudne; layout liczy się raz na klatkę tylko dla brudnych poddrzew.
- Jednostki:** punkty logiczne rozdzielczości bazowej, pomnożone przez `ui().scale()` (ustawienie gracza 75–200%). Pozycje końcowe zaokrąglane do pikseli, by tekst był ostry.

18.6 Stan i powiązania

UI odświeża się samo, bez ręcznego ustawiania tekstów:

```
State<Integer> coins = State.of(0);
label(coins.map(c -> tr("hud.coins", c)));          // odświeża się przy
coins.set(...)
slider(0, 1).bind(settings.musicVolume());          // powiązanie dwukierunkowe
listView(quests, quest -> row(label(quest.name()), spacer(),
label(quest.progress())));
```

`State<T>` (obserwowalna wartość), `ListState<T>` (lista z powiadomieniami o zmianach i minimalnymi aktualizacjami węzłów), `Computed` (wartość wyliczana z innych).

Subskrypcje należą do węzła i znikają z nim.

18.7 Motywy

- `Theme` : style dla każdego typu widgetu i stanu (`normal` , `hover` , `pressed` , `focused` , `disabled` , `checked`): tło (`StyleBox` : kolor z zaokrągleniem, obramowaniem i cieniem; `nine-patch` z tekstury; brak), font, rozmiar, kolory, padding, odstępy, dźwięki kliknięcia i hovera.
- Warianty: `button(...).variant("danger")` . Dziedziczenie: motyw ustawiony na węźle dotyczy całego poddrzewa. Nadpisanie jednego węzła: `.style(s -> s.textColor(Color.RED))` .
- Wbudowany nowoczesny motyw jasny i ciemny, szybka zmiana akcentu: `Theme.DEFAULT.withAccent(Color.hex("#7c5cff"))` , oraz motyw pixel-art.
- Płynne przejścia między stanami (np. kolor hovera przez 0,1 s) włączone domyślnie.

18.8 Fokusz i sterowanie padem

Graf fokusu powstaje automatycznie z geometrii (najbliższy sąsiad w danym kierunku), można go nadpisać (`focusNeighbor(Direction.DOWN, node)`). Akcje `gulp:ui_accept` , `ui_cancel` , `ui_up` , `ui_down` , `ui_left` , `ui_right` , `ui_next_tab` , `ui_prev_tab` są wbudowane i przypisane do klawiatury i pada. `Screen.defaultFocus()` wskazuje pierwszy element; ramkę fokusu rysuje motyw. Każde menu musi być w pełni obsługiwalne samym padem.

18.9 Ekran

- `ui().open(screen)` (zastępuje stos), `push(screen)` , `pop()` , `current()` . Cykl życia: `build()` , `onOpen()` , `onClose()` , `onBack()` (domyślnie `pop`).
- Ustawienia ekranu: `pausesGame` , `blocksGameplayInput` , `dimBackground` , przejście wejścia i wyjścia (`ScreenTransition.fade` , `slideUp` , `scale`).

- Eventy: `ScreenOpenEvent` (anulowalny), `ScreenCloseEvent`.

18.10 HUD i UI w świecie

- HUD to stała warstwa ekranowa (`ui().hud()`), do której moduły dodają węzły z kotwicami: `ui().hud().add(this, label(coins.map(...)).anchor(Anchor.TOP_LEFT).offset(8, 8))`. Węzły znikają z modułem-właścicielem. Silnik nie narzuca żadnych gotowych elementów HUD — wszystko składa się z tych samych widgetów.
- Komponent `WorldUi` przypina węzeł UI do encji (np. pasek życia nad wrogiem): pozycja śledzi encję przez kamerę, węzeł chowa się poza ekranem, opcjonalnie skaluje z zoomem.

18.11 `Overlay` — rysowanie natychmiastowe

Do prostych, ogólnych rzeczy bez budowania drzewa, wywoływane co klatkę:

```
ui().overlay().draw(this, (draw, screen) -> {
    draw.text("Pauza", screen.center(), TextStyle.of(32), TextAlign.CENTER);
    draw.text("x: " + player.position().x(), 8, 8);
    draw.rect(screen.anchor(Anchor.BOTTOM_RIGHT, -108, -28, 100, 20),
        Color.BLACK.withAlpha(0.5f));
});
```

`screen` daje rozmiar ekranu w punktach logicznych i pomocnicze `center()`, `anchor(...)`, więc nawet tutaj nie trzeba liczyć pozycji względem rozdzielczości. `Overlay` służy też do debugowania.

18.12 Przeciąganie, podpowiedzi, menu kontekstowe

`.draggable(payload)` i `.dropTarget(accept, onDrop)` z podglądem przeciąganego elementu; `.tooltip("tekst")` lub `.tooltip(node)` z opóźnieniem z motywu; `.contextMenu(menu(item("Usuń", ...), item("Podziel", ...)))`.

18.13 Dostępność i debug

- Skalowanie UI przez gracza, wariant wysokiego kontrastu motywu, minimalny rozmiar tekstu. Integracja z czytnikami ekranu po 1.0 (decyzja w sekcji 23).
- Inspektor UI (skrót w trybie deweloperskim) rysuje ramki węzłów, pokazuje rozmiary, flagi i kotwice po najechaniu kursorem.

19. Dane: zapis gry, preferencje, lokalizacja, sieć

Zapis gry działa na zasadzie z Minecrafta: trwałe encje, komponenty, zmiany kafelków i dane modułów zapisują się automatycznie w binarnym formacie z tagami, z wersjonowaniem i migracjami. Gra dopisuje własne dane przez eventy.

19.1 Zapisy gry

- `saves().slot("slot1")` zwraca `SaveSlot`: `save()` i `load()` (oba `Promise`), `delete()`, `exists()`, `metadata()` (nazwa, data, czas gry, miniatura ze zrzutu ekranu, wersja). `saves().list()` zwraca wszystkie sloty.
- Co zapisuje się samo: światy z `persistent`, ich trwałe encje z trwałymi komponentami (codeki generowane z `@Save`), `DataContainer` encji, światów i modułów, kafelki ze stanem, zmiany mapy względem mapy źródłowej (delta) albo całe chunki w światach generowanych.
- Własne dane: `GameSaveEvent` i `GameLoadEvent` dają `DataContainer` slotu.
- Format: tagowany format binarny (typy: liczby, stringi, listy, mapy, tablice liczb), kompresja przez platformę (`Deflater` na desktopie, `CompressionStream` na webie). Światy z chunkami w plikach regionów (32 × 32 chunki na plik).
- Wersjonowanie: `GameSettings.saveVersion(n)` i migracje `saves().migration(fromVersion, data -> ...)` wykonywane po kolei przy wczytywaniu starszego zapisu.
- Bezpieczeństwo danych: zapis atomowy (plik tymczasowy i podmiana) na desktopie, transakcje `IndexedDB` na webie; opcjonalny autozapis co N minut i zawsze przy ukryciu karty na webie (zamknięcie karty nie gwarantuje `onDisable` ani `onStop`); eksport i import slotu jako plik (na webie pobieranie i wgrywanie).

19.2 Preferencje

`preferences()` to mały magazyn klucz–wartość niezależny od slotów: `getFloat("audio.music", 0.8f)`, `set(...)`, zapis automatyczny. Silnik sam przechowuje tam ustawienia wbudowane: tryb okna, `VSync`, skalę UI, głośności szyn, przypisania akcji i język. Przykład `showcase` zawiera gotowy ekran ustawień do skopiowania.

19.3 Lokalizacja

- Pliki `assets/<namespace>/lang/<locale>.json` (klucze płaskie lub zagnieżdżone).
- `tr("hud.coins", 5)` i `Text.translatable(...)` z formatowaniem: `{0}`, nazwane argumenty i liczba mnoga, np. `{count, plural, one {# moneta} few {# monety} many {# monet} other {# monety}}`. Reguły liczby mnogiej dla popularnych języków

(w tym polskiego) wbudowane.

- Łańcuch zapasowy: pełny locale → język → język domyślny gry → sam klucz (z ostrzeżeniem w logu).
- `translations().setLocale("pl_pl")` wywołuje `LocaleChangeEvent`; teksty UI oparte na `Text.translatable` odświeżają się same. Język startowy z systemu lub preferencji.
- Proste formatowanie liczb i dat zgodne z językiem.

19.4 Sieć

- `http().get(url)`, `post(url, body)`, `request(HttpRequest)` zwracają `Promise<HttpResponse>` (`status`, nagłówki, `text()`, `json()`, `bytes()`), z limitem czasu. Na webie obowiązują zasady CORS — Javadoc musi o tym mówić.
- `WebSocket`: `connect(url)`, `send(text | bytes)`, handlers `onOpen`, `onMessage`, `onClose`, `onError`, zawsze na głównym wątku.
- `platform().openUrl(url)`.
- Pełny multiplayer (synchronizacja stanu, predykcja) poza 1.0.

20. Jakość: diagnostyka, błędy, wątki, wydajność, testy

Framework ma być odporny jak serwer Paper: błąd jednego modułu lub komponentu nie wyraca gry, wszystko jest mierzalne, a każda funkcja ma testy działające bez okna.

20.1 Narzędzia deweloperskie

- Nakładka F3: FPS i wykres czasu klatki, TPS i czas ticka, liczba encji, chunków, cząstek, dźwięków, wywołań rysowania i zmian tekstur, pamięć (desktop), pozycja kursora w świecie i numer kafelka.
- Przełączniki (F3 + klawisz lub `/debug`): kształty kolizji, siatka nawigacji i ścieżki, granice chunków, światła, inspektor UI, inspektor encji (kliknięta encja pokazuje komponenty, tagi, dane i ich wartości na żywo).
- `debug().draw()`: `line`, `rect`, `circle`, `arrow`, `text` w świecie, z czasem życia w tickach. W buildzie produkcyjnym to no-op.
- Profiler: czas każdego modułu, listenera, zadania i typu komponentu; własne sekcje `try (var s = debug().section("ai")) {...}; /profile start|stop` zapisuje raport do konsoli i pliku.
- Tryb deweloperski (`runDesktop`, `runWeb`) vs produkcyjny (`package*`): w produkcji wyłączone są konsola, inspektory i hot reload, chyba że gra je włączy.

20.2 Logi i raporty awarii

- Logger per moduł z poziomami; na deskkopie `logs/latest.log` z rotacją archiwów, na webie konsola przeglądarki.
- Raport awarii w `crash-reports/crash-<data>.txt` : stos wywołań, lista modułów i ich stan, system, GPU, wersje, ostatnie linie logu. Na webie ten sam raport w nakładce błędu z przyciskiem kopiowania.

20.3 Polityka błędów

Sytuacja	Zachowanie
Wyjątek w listenerze, zadaniu, komponencie	Log z właścicielem; po 3 błędach z rzędu komponent lub zadanie jest wyłączane
Wyjątek w <code>onEnable</code> modułu	Moduł <code>FAILED</code> , zależne moduły wyłączone, gra działa
Brakujący zasób	Zastępczy zasób (tekstura w fioletową szachownicę, cisza, font domyślny) + ostrzeżenie
Błąd kompilacji shadera	Shader zastępczy + log z numerem linii
Utrata kontekstu WebGL	Automatyczne odtworzenie zasobów GPU
Błąd fatalny (brak GL, brak pamięci)	Raport awarii i czytelny ekran błędu zamiast cichego zamknięcia
Wywołanie API spoza głównego wątku	W trybie deweloperskim wyjątek z nazwą metody i wątku

20.4 Model wątków

Jeden wątek główny wykonuje ticki, renderowanie, eventy i UI. Poza nim działają tylko: dekodowanie zasobów, generowanie chunków, duże obliczenia ścieżek i zapis na dysk. Metody API bezpieczne wątkowo mają adnotację `@ThreadSafe` ; reszta jest tylko dla wątku głównego. Na webie wszystko działa na jednym wątku, a praca „asynchroniczna” jest dzielona na kawałki między klatkami.

20.5 Budżety wydajności

Miara	Cel
Tick typowej sceny przy 60 TPS	≤ 4 ms

Miara	Cel
Renderowanie klatki typowej sceny	≤ 8 ms
Alokacje na klatkę w stanie ustalonym (rdzeń)	0
Sprite'y przy 60 FPS	10 000 web, 50 000 desktop
Aktywne encje przy 60 TPS	5 000 web, 20 000 desktop
Start do pierwszej klatki (desktop)	< 2 s
Rozmiar Wasm przykładu <code>showcase</code> bez zasobów	< 5 MB po kompresji gzip

Sceny wydajnościowe w `examples/bench` (sprite'y, encje, cząsteczki, UI, mapa) są uruchamiane w CI; testy alokacji działają na backendzie headless.

20.6 Testy

- Jednostkowe: każdy podsystem rdzenia na backendzie headless (JUnit Jupiter + AssertJ).
- Testy gier przez `gulp-test`:

```
var game = GameTestHarness.start(new CoinGame());
game.tick(60);
game.press(CoinGame.JUMP, 5);
assertThat(game.world().query().tag("coin").count()).isEqualTo(9);
```

- Testy wizualne: na Linuksie w CI z programowym OpenGL (Mesa llvmpipe) i Xvfb, porównanie zrzutów z obrazami wzorcowymi z tolerancją.
- Web: testy jednostkowe uruchamiane przez TeaVM w przeglądarce headless oraz test dymny, który ładuje `showcase`, klika przez ekrany i sprawdza brak błędów w konsoli.
- API: `checkApiUsage` w przykładach, a po 1.0 porównanie publicznego API z poprzednią wersją (zmiany łamiące blokują build).
- CI: macierz Linux, Windows, macOS; kroki: formatowanie, testy, build web, testy wizualne, testy dymne web, benchmarki.

21. Przykładowa gra

Poniższa gra (platformówka ze zbieraniem monet) to test akceptacyjny API: kiedy ten kod kompiluje się i działa na desktopie i w przeglądarce bez zmian, rdzeń frameworka spełnia swoje założenia. Cała gra to około 100 linii, bez jednej macierzy i bez ręcznego liczenia pozycji UI. Trafia do `examples/platformer`.

```

public final class CoinGame extends Game {
    public static InputAction MOVE_LEFT, MOVE_RIGHT, JUMP;
    public static CollisionLayer PICKUP;
    public static EntityType PLAYER, COIN;
    public static Sound PICKUP_SOUND;
    public static final Key COINS = Key.of("coins", "coins");

    @Override public String id() { return "coins"; }

    @Override public void configure(GameSettings s) {
        s.title("Coin Hunter").baseResolution(480, 270)
        .stretchMode(StretchMode.VIEWPORT).integerScaling(true)
        .modules(new HudModule(), new PlayerModule(), new CoinModule());
    }

    @Override public void onLoad() {
        var r = registries();
        MOVE_LEFT = r.register(Registries.INPUT_ACTION,
        InputAction.builder(key("move_left")).bind(Keys.A, Keys.LEFT,
        GamepadAxis.LEFT_X.negative()).build());
        MOVE_RIGHT = r.register(Registries.INPUT_ACTION,
        InputAction.builder(key("move_right")).bind(Keys.D, Keys.RIGHT,
        GamepadAxis.LEFT_X.positive()).build());
        JUMP = r.register(Registries.INPUT_ACTION,
        InputAction.builder(key("jump")).bind(Keys.SPACE,
        GamepadButton.SOUTH).build());
        PICKUP = r.register(Registries.COLLISION_LAYER,
        CollisionLayer.of(key("pickup")));
        PICKUP_SOUND = r.register(Registries.SOUND,
        Sound.builder(key("pickup"))
        .file(GameAssets.Sounds.PICKUP).pitchRange(0.9f,
        1.1f).maxInstances(4).build());
    }

    @Override public void onStart() {
        worlds().register("level1", WorldSource.ldtk(GameAssets.Maps.LEVEL1)
        .spawn("Player", PLAYER).spawn("Coin", COIN));
        ui().open(new MainMenu()); // z sekcji 18.1
    }
}

@ModuleInfo(id = "player")
public final class PlayerModule extends GameModule {
    @Override public void onLoad() {
        CoinGame.PLAYER = registries().register(Registries.ENTITY_TYPE,
        EntityType.builder(key("player")))

```

```

        .size(0.8f, 1.6f)
        .component(() -> new SpriteComponent(GameAssets.Sprites.PLAYER))
        .component(() -> new Animator(GameAssets.Animations.PLAYER))
        .component(() -> new Mover().coyoteTicks(6).jumpBufferTicks(6))
        .component(PlayerController::new)
        .tags("player").build());
    }
}

public final class PlayerController extends Component {
    private Mover mover;

    @Override protected void onSpawn() {
        mover = entity().get(Mover.class);
        entity().get(Animator.class).auto()
            .when(() -> !mover.isOnFloor(), "jump")
            .when(() -> mover.velocity().x() != 0, "run")
            .otherwise("idle");
    }

    world().camera().follow(entity()).smoothing(6).limits(world().tileMap().bounds());
}

    @Override protected void onTick() {
        Input input = input();
        float x = input.axis(CoinGame.MOVE_LEFT, CoinGame.MOVE_RIGHT) * 7f;
// jednostki na sekundę
        if (input.justPressed(CoinGame.JUMP) && mover.canJump())
mover.jump(12f);
        mover.moveAndSlide(mover.velocity().withX(x));
// grawitacja dodawana przez Mover
        if (x != 0) entity().setFlipX(x < 0);
    }
}

@ModuleInfo(id = "coin", dependsOn = {"player", "hud"})
public final class CoinModule extends GameModule {
    @Override public void onLoad() {
        CoinGame.COIN = registries().register(Registries.ENTITY_TYPE,
EntityType.builder(key("coin"))
        .size(0.5f, 0.5f)
        .component(() -> new
Animator(GameAssets.Animations.COIN).play("spin"))
        .component(() -> new Trigger().layer(CoinGame.PICKUP))
        .tags("coin").build());
    }
}

```

```

@Override public void onEnable() {
    on(TriggerEnterEvent.class, e -> {
        Entity coin = e.trigger().entity();
        if (!coin.tags().has("coin") || !e.other().tags().has("player"))
            return;

        coin.get(Trigger.class).setEnabled(false);
        coin.world().playSound(coin.position(), CoinGame.PICKUP_SOUND);
        Tweens.parallel(
            Tweens.by(coin, Props.Y, -1f, 0.25f).ease(Ease.OUT_QUAD),
            Tweens.to(coin, Props.ALPHA, 0f, 0.25f))
            .onComplete(coin::remove)
            .start();
        State<Integer> coins = require(HudModule.class).coins();
        coins.set(coins.get() + 1);
    });
}

@ModuleInfo(id = "hud")
public final class HudModule extends GameModule {
    private final State<Integer> coins = State.of(0);
    public State<Integer> coins() { return coins; }

    @Override public void onEnable() {
        ui().hud().add(this, row(
            image(GameAssets.Sprites.COIN_ICON),
            label(coins.map(c -> tr("hud.coins", c))))
            .gap(4).anchor(Anchor.TOP_LEFT).offset(8, 8));
        on(GameSaveEvent.class, e -> e.data().set(CoinGame.COINS,
            DataType.INT, coins.get()));
        on(GameLoadEvent.class, e ->
            coins.set(e.data().getOrDefault(CoinGame.COINS, DataType.INT, 0)));
    }
}

```

`assets/coins/lang/pl_pl.json` zawiera `"hud.coins": "{0, plural, one {# moneta} few {# monety} many {# monet} other {# monety}}"`. Mapa `level1.ldtk` ma encje `Player` i `Coin` oraz warstwę `IntGrid` z kolizjami.

22. Roadmapa

Trzynaście etapów od pustego repozytorium do wersji 1.0, wykonywanych po kolei; każdy kończy się kamieniem milowym, który da się uruchomić i przetestować. Web pojawia się w etapie 3, żeby ograniczenia TeaVM wyszły na jaw, zanim API się utrwali. Zadania odhacza

Claude po spełnieniu definicji ukończenia z sekcji 1.

Etap	Cel	Kamień milowy
0. Fundamenty	Repo, build, CI, SPI platformy, okno	Puste okno na 3 systemach, <code>check</code> zielony
1. Rdzeń	Engine, Game, moduły, eventy, scheduler, rejestry, dane	Test headless gry z dwoma modułami przechodzi
2. Matematyka i grafika	Math, GL, batcher, <code>Draw</code> , kamera, <code>Display</code>	10 000 sprite'ów przy 60 FPS na desktopie
3. Web	Backend TeaVM, zasoby asynchroniczne, plugin Gradle	Ta sama scena w Chrome, Firefox, Safari
4. Zasoby i tekst	Atlasy, fonty, tekst, hot reload	Polski tekst we wszystkich fontach, hot reload działa
5. Input i audio	Akcje, urządzenia, dźwięk	Sterowanie klawiaturą, myszą i padem na obu platformach
6. Świat i encje	Encje, komponenty, mapy, chunki, importy	Poziom z LDtk i nieskończony świat proceduralny
7. Fizyka, nawigacja, AI	Kinematyka, bryły sztywne, A*, AI	Grywalna platformówka i stabilna piaskownica fizyki
8. Animacje i efekty	Tweeny, animacje, cząsteczki, światło	Scena z efektami w showcase
9. UI	Layout, widgety, motywy, stan, fokus, ekrany	Galeria UI obsługiwana samym padem
10. Dane	Zapisy, preferencje, lokalizacja, sieć	Zapis i wczytanie gry w przeglądarce i na desktopie
11. Jakość i narzędzia	Debug, profiler, pakowanie, benchmarki	Gra z szablonu spakowana na 3 systemy i web
12. Wydanie	Dokumentacja, audyt parytetu, publikacja	Wersja 1.0

Etap 0 — Fundamenty

- ☐ Repo: licencja, `README.md`, `CLAUDE.md` (skrót sekcji 1 + komendy), `docs/spec.md` (kopia tego dokumentu), `docs/decisions/0001-architektura.md`, `CHANGELOG.md`.

- ☐ Gradle 9: wszystkie moduły z sekcji 6 jako puste szkielety, `libs.versions.toml`, `build-logic` (toolchain Java 25, Spotless, JUnit, JSpecify, `-Werror`).
- ☐ CI (GitHub Actions): macierz Linux, Windows, macOS; zadanie builda web jako zaczepka na etap 3.
- ☐ `gulp-platform`: wszystkie interfejsy SPI z sekcji 7 z Javadoc.
- ☐ `gulp-backend-headless`: pętla sterowana ręcznie (`step(n)`).
- ☐ `gulp-backend-desktop`: okno GLFW, kontekst OpenGL 3.3 core, czyszczenie kolorem, zamknięcie, restart JVM z `-XstartOnFirstThread` na macOS.
- ☐ Launcher: `Gulp.launch(new MyGame())` dla desktopu i headless.
- ☐ Kamień milowy: okno z kolorem tła na 3 systemach (Linux w CI przez Xvfb), `./gradlew check` zielony.

Etap 1 — Rdzeń

- ☐ `Engine`, `Gulp`, pętla ze stałym krokiem (akumulator, limit 5 ticków nadrabiania, `alpha` interpolacji), pauza i `timeScale`.
- ☐ `Game` i `GameSettings`.
- ☐ System modułów: `@ModuleInfo`, walidacja w procesorze adnotacji i przy starcie, sortowanie topologiczne z błędem cyklu, cykl życia, śledzenie własności, włączanie i wyłączanie w trakcie gry, stan `FAILED`.
- ☐ Eventy: `Event`, `Cancellable`, `HandlerList`, priorytety, dispatch generowany przez procesor, subskrypcje lambda, subskrypcje przypięte do obiektu, polityka wyjątków, dziedziczenie eventów.
- ☐ Scheduler: zadania, `TaskRunnable`, `Sequence`, `Promise`, `async().thenSync()`, pauza i `realtime()`.
- ☐ `Key`, `Registry`, zamrażanie, rejestry wbudowane (puste typy).
- ☐ `Services`.
- ☐ Dane: parser i zapis JSON, parser podzbioru YAML, `Config`, `Codec` + generowanie dla `@Serializable`, `DataContainer`, `DataType`.
- ☐ Logger (plik na desktopie), `Pool`, kolekcje prymitywne.
- ☐ Komendy: rejestracja, argumenty typowane, podpowiedzi, konsola w terminalu (konsola w grze w etapie 9).
- ☐ Kamień milowy: test headless — gra z dwoma modułami (zależność, listener, `every(20)`, odczyt konfiguracji); wyłączenie modułu anuluje jego zadania i listenery.

Etap 2 — Matematyka i grafika

- ☐ Pakiet `math` w całości z sekcji 11, testy właściwości.

- ☐ `GL` i implementacja desktop; `Texture`, `TextureRegion`, `Pixmap`, dekodowanie `stb_image`.
- ☐ `Shader` z include'ami i tłumaczeniem nagłówka, `Material`, `BlendMode`.
- ☐ `Batcher` i `Draw`: obrazy, kształty z antyaliasingiem, transformacje, `clip`, `into(fbo)`; statystyki.
- ☐ `FrameBuffer`, `Mesh2D`, zrzut ekranu.
- ☐ `Camera` (pozycja, zoom, rotacja, konwersje współrzędnych).
- ☐ `Display`: rozdzielczość bazowa, `StretchMode`, `AspectMode`, skalowanie całkowite, HiDPI, pasy.
- ☐ `RenderLayer`, eventy renderowania.
- ☐ Kamień milowy: scena z 10 000 ruchomych sprite'ów przy 60 FPS; zmiana rozmiaru okna zachowuje układ według trybów; testy wizualne z obrazami wzorcowymi.

Etap 3 — Web

- ☐ `gulp-backend-web`: konfiguracja TeaVM (Wasm GC + fallback JS), szablon HTML z ekranem ładowania.
- ☐ Pętla `requestAnimationFrame`, `GL` na WebGL2, canvas z DPI i zmianą rozmiaru, pełny ekran.
- ☐ Surowe zdarzenia klawiatury i myszy na obu platformach (pełny input w etapie 5).
- ☐ `PlatformFiles` przez `fetch`, manifest zasobów, dekodowanie obrazów w przeglądarce, wykonawca kooperacyjny.
- ☐ `Assets`: klucze, `load / get`, grupy, liczenie referencji, grupa `startup` i domyślny ekran ładowania.
- ☐ Plugin Gradle: `runDesktop`, `runWeb` (serwer z automatycznym odświeżaniem), `buildWeb`.
- ☐ Test dymny w przeglądarce headless w CI.
- ☐ Kamień milowy: scena z etapu 2 działa bez zmian w Chrome, Firefox i Safari; rozmiar paczki zmierzony i zapisany.

Etap 4 — Zasoby, tekst i fonty

- ☐ Packer atlasów (narzędzie + pakowanie w locie w trybie deweloperskim), regiony po ścieżce.
- ☐ `generateAssetKeys`.
- ☐ Generowanie MSDF w buildzie, shader `Msdffont` z obrysem i cieniem.
- ☐ `BitmapFont`, `DynamicFont` (FreeType na desktopie, Canvas2D na webie), cache glifów, fallbacki.

- ☐ `TextStyle` , `TextLayout` , `Text` , parser markupu z efektami, `Draw.text` .
- ☐ Podstawowe `tr()` i pliki językowe (pełna lokalizacja w etapie 10).
- ☐ Hot reload na desktopie, `AssetReloadEvent` ; resource packi; własne `AssetLoader` .
- ☐ Kamień milowy: ekran tekstu z polskimi znakami we wszystkich typach fontów i efektami markupu; zmiana tekstury i tekstu widoczna bez restartu.

Etap 5 — Input i audio

- ☐ Stany per tick, eventy wejścia, rejestr akcji, `ActionSet` , `axis` , `vector` , martwe strefy.
- ☐ Zmiana przypisań, przechwytywanie, podstawowe `Preferences` do ich zapisu, ikony przypisań.
- ☐ Kursory i tryby, schowek, tekst z IME, dotyk i gesty (web).
- ☐ Gamepady: desktop (GLFW + mapowania), web (Gamepad API), podłączanie w locie, wibracje gdzie możliwe.
- ☐ Audio desktop (OpenAL): głosy, szyny, warianty i limity dźwięków, dźwięk pozycyjny, muzyka strumieniowana z punktami pętli, crossfade, filtry.
- ☐ Audio web (WebAudio) z tym samym zakresem i odblokowaniem po geście; strumień PCM.
- ☐ Kamień milowy: ekran input/audio w showcase działa z klawiaturą, myszą i padem na obu platformach; zmienione przypisania przetrwają restart.

Etap 6 — Świat i encje

- ☐ `Worlds` , `World` , `WorldSettings` , `Location` , przejścia.
- ☐ `Entity` , `EntityType` , `Component` z cyklem życia, hierarchia, tagi, dane, interpolacja pozycji, `PauseMode` .
- ☐ Przechowywanie komponentów, `EntityQuery` , siatka przestrzenna, eventy encji (w tym klik, hover, ekran).
- ☐ Komponenty: `SpriteComponent` , `Lifetime` , `Follow` , `WorldText` , `Interactable` .
- ☐ `TileMap` : orientacje, warstwy, `TileType` , kształty kolizji, animowane kafelki, ticki kafelków, `TileState` .
- ☐ Chunki: ładowanie wokół kamer i obszarów, `ChunkGenerator` poza tickiem, cache siatek renderowania.
- ☐ `Terrain` (16 i 47 kafelków), importy Tiled i LDtk, `ObjectSpawner` , `Parallax` .
- ☐ Kamera: podążanie, wygładzanie, granice, martwa strefa, wstrząsy, `zoomTo` , `panTo` , wiele kamer.
- ☐ Kamień milowy: `examples/topdown` z nieskończonym światem z szumu i poziom platformówki z LDtk renderowany poprawnie.

Etap 7 — Fizyka, nawigacja i AI

- ☐ Warstwy kolizji, `Collider`, faza szeroka i SAT.
- ☐ `Mover` w pełnym zakresie z sekcji 15.2; `Trigger`; `EntityLandEvent`.
- ☐ Zapytania: `raycast`, `raycastAll`, `shapeCast`, `overlap*`.
- ☐ Bryły sztywne: ciała, kształty, solver z podkrokami, usypianie, CCD, złącza, eventy kontaktów, `PreCollideEvent`.
- ☐ `NavGrid`, `A*`, JPS, pola przepływu, `A*` na grafie, `NavAgent`, `Path`, `PathFollower`.
- ☐ `StateMachine`, `BehaviorTree`, `Steering`.
- ☐ Rysowanie debug wszystkich powyższych.
- ☐ Kamień milowy: `examples/platformer` grywalny (kod z sekcji 21 działa); piaskownica fizyki (stosy skrzyń, łańcuchy ze złączami) stabilna; wrogowie w topdown znajdują drogę.

Etap 8 — Animacje, tweeny, cząsteczki, światło

- ☐ `Property`, `Props`, `Tweens` w całości z efektami gotowymi, `Timeline`.
- ☐ `SpriteAnimation`, `AnimationSet`, import Aseprite, `Animator` z `auto()`.
- ☐ Cząsteczki: emitery, krzywe, gradienty, pod-emitery, kolizje, JSON z hot reloadem, `ParticleEmitter`.
- ☐ Światło 2D z cieniami od occluderów i kafelków; łańcuch post-process i efekty wbudowane; materiały gotowe.
- ☐ Kamień milowy: scena „juice” w showcase (błyski, wstrząsy, cząsteczki, światło) na obu platformach.

Etap 9 — UI

- ☐ `Node`, silnik layoutu (pomiar, rozmieszczenie, kotwice, unieważnianie), skala UI.
- ☐ Wszystkie kontenery i widgety z sekcji 18.3 i 18.4.
- ☐ `Theme` z motywami jasnym, ciemnym i pixel-art, przejścia stanów.
- ☐ `State`, `ListState`, `Computed`, powiązania.
- ☐ Fokus i nawigacja padem, akcje `ui_*`.
- ☐ Ekrany: stos, przejścia, `pausesGame`, `blocksGameplayInput`.
- ☐ HUD, `WorldUi`, `Overlay`, przeciąganie, podpowiedzi, menu kontekstowe, powiadomienia.
- ☐ Kolejność obsługi wejścia (UI przed grą); konsola deweloperska w grze; inspektor UI.
- ☐ Kamień milowy: `examples/ui-gallery` ze wszystkimi widжетami w pełni obsługiwalna padem; menu i ustawienia w showcase bez żadnych współrzędnych w kodzie gry.

Etap 10 — Dane

- ☐ Tagowany format binarny + kompresja; `SaveStore`, sloty, metadane, miniatury.
- ☐ Zapis światów, encji, komponentów, kafelków i chunków (pliki regionów); migracje; autozapis; eksport i import.
- ☐ `Preferences` w całości z ustawieniami wbudowanymi.
- ☐ Lokalizacja: liczba mnoga, formatowanie, łańcuch zapasowy, `LocaleChangeEvent`.
- ☐ `Http`, `WebSocket`, `openUrl`.
- ☐ Kamień milowy: testy zapisu i odczytu wszystkich danych trwałych; platformówka zapisuje postęp w przeglądarce i na desktopie.

Etap 11 — Jakość i narzędzia

- ☐ Nakładka F3, przełączniki, inspektor encji, `debug().draw()`, profiler z raportem.
- ☐ Raporty awarii, rotacja logów, testy polityki błędów z sekcji 20.3.
- ☐ Plugin Gradle w całości: `packageDesktop` (jlink + jpackage), `packageWeb`, `packAssets`, `checkApiUsage`.
- ☐ `examples/bench` z programami w CI, testy alokacji, testy wizualne, testy dymne web.
- ☐ Szablon nowego projektu gry.
- ☐ Kamień milowy: projekt z szablonu pakuje się na Windows, macOS, Linux i web po jednej komendzie na platformę.

Etap 12 — Wydanie

- ☐ Dokumentacja: pierwsze kroki, przewodnik po każdym podsystemie, przewodnik „dla znających Bukkit”, Javadoc online.
- ☐ Dopracowane przykłady: `showcase`, `platformer`, `topdown`, `ui-gallery`, `bench`.
- ☐ Audyt: każdy wiersz tabel z sekcji 9 i 10 potwierdzony przykładem lub testem.
- ☐ Przegląd i zamrożenie API, publikacja 0.x w Maven Central, włączenie kontroli zgodności API, wydanie 1.0 po zebraniu uwag.

23. Decyzje

Cztery decyzje są podjęte, a dwanaście czeka na potwierdzenie; każda otwarta ma propozycję, którą Claude stosuje, jeśli do wskazanego etapu nie zapadnie inna decyzja. Po zmianie statusu Claude zapisuje decyzję jako ADR w `docs/decisions/`.

Decyzja	Opcje	Propozycja	Potrzebna przed etapem	Status
Organizacja logiki gry	drzewo węzłów / encje + eventy / hybryda	Encje + eventy jak w Minecrafcie	—	Podjęta
Tworzenie scen i UI	tylko kod / kod + pliki / edytor	Tylko kod, deklaratywne API	—	Podjęta
Warstwa platformy	własna (LWJGL3 + TeaVM) / libGDX	Własna	—	Podjęta
Nazwa projektu	Tessera / Gulp	Gulp (wybór właściciela). Uwaga: kolizja z narzędziem gulp.js utrudni wyszukiwanie; sprawdzić grupę Maven i domenę	0	Podjęta
Licencja	Apache 2.0 / MIT / GPL	Apache 2.0	0	Otwarta
Kierunek osi Y	w dół wszędzie (Godot, Tiled, LDTk) / w górę w świecie (libGDX)	W dół wszędzie — zgodnie z edytorami map i UI	2	Otwarta
Jednostka świata	1 = kafelek / 1 = piksel	1 = kafelek (rozmiary niezależne od rozdzielczości grafiki)	2	Otwarta
Domyślne TPS	60 / 20 (jak Minecraft)	60, konfigurowalne	1	Otwarta
Format konfiguracji	podzbiór YAML / TOML / JSON5	Podzbiór YAML (znajomy z Bukkita)	1	Otwarta
Domyślny cel web	Wasm GC / JS	Wasm GC z automatycznym fallbackiem JS	3	Otwarta
Fizyka brył sztywnych	własny solver / port jbox2d (zlib)	Własny; jbox2d jako plan awaryjny	7	Otwarta

Decyzja	Opcje	Propozycja	Potrzebna przed etapem	Status
Pełne kształtowanie tekstu (RTL, pisma złożone)	w 1.0 / po 1.0	Po 1.0	4	Otwarta
Mody jako zewnętrzne JAR-y (desktop)	w 1.0 / po 1.0	Po 1.0	12	Otwarta
Czytniki ekranu	w 1.0 / po 1.0	Po 1.0	9	Otwarta
Animacje szkieletowe	własne / DragonBones / Spine (wymaga licencji) / brak	Po 1.0, własny format lub DragonBones	12	Otwarta
Android, iOS, multiplayer	w 1.0 / później	Później; architektura ich nie blokuje	12	Otwarta

Ryzyka

Ryzyka techniczne, które Claude sprawdza w pierwszym etapie, w którym mogą wyjść na jaw; każde ma plan awaryjny.

Ryzyko	Kiedy sprawdzić	Plan awaryjny
Apple oznaczyło OpenGL na macOS jako przestarzały i może go usunąć	Etap 0 i każde wydanie macOS	Warstwa <code>gl</code> pozwala dodać backend przez ANGLE (OpenGL ES na Metalu) bez zmian w <code>core</code>
Safari: obsługa Wasm GC i dekodowania OGG w WebAudio	Etap 3 (Wasm), etap 5 (audio)	Fallback JS; dodatkowy eksport audio do formatu obsługiwanego przez Safari w <code>packAssets</code>
Luki w bibliotece standardowej TeaVM (np. część <code>java.time</code> , <code>String.format</code> , współbieżność)	Od etapu 1, build web w CI od etapu 3	W <code>core</code> tylko sprawdzone klasy JDK; własne zamienniki w <code>core.internal</code>
Wysiłek własnego solvera brył sztywnych	Etap 7	jbox2d (zlib) za tym samym API (decyzja w tabeli wyżej)

Ryzyko	Kiedy sprawdzić	Plan awaryjny
Rozmiar paczki Wasm rośnie ponad budżet	Pomiar w każdym etapie od 3	Profil rozmiaru TeaVM, dzielenie rzadko używanych podsystemów
GLFW może nie obsługiwać wibracji padów	Etap 5	Wibracje jako opcjonalne (<code>gamepad.supportsRumble()</code>); rozważyć SDL3, jeśli LWJGL go udostępnia
Nazwa „Gulp” myli się z gulp.js	Przed publikacją (etap 12)	Pełna nazwa w materiałach, np. „Gulp Engine”, i unikalna grupa Maven